



Title: SmartRust AI for Wheat Disease Detection and Prevention

FINAL YEAR PROJECT

PHASE-I

Submitted by

Name: M. Faseeh Ur Rehman 70127022

Name: Rehan Tariq 70126162

Name: Fahad Imran 70127070

Project Supervisor:

Mr. M. Almas Manzoor

BACHELOR OF SCIENCE IN SOFTWARE ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING, UNIVERSITY OF LAHORE

FINAL YEAR PROJECT PHASE-I DOCUMENTATION

STATEMENT OF SUBMISSION

Submitted to the University of Lahore in partial fulfillment of the requirement for the award of degree of Bachelors of Science in Software Engineering (BSSE)

By:

• M. Faseeh Ur Rehman	70127022	_____
• Rehan Tariq	70126162	_____
• Fahad Imran	70127070	_____

Project Advisor: _____

Mr. M. Almas Manzoor

Lecturer at University of Lahore

Department of Software Engineering

Dedication

This work is submissively devoted to the hardworking growers around the world, whose grim sweats ensure food security for all. It's also devoted to agrarian experimenters and originators seeking to make husbandry more effective and sustainable. Incipently, to our instructors, families, and musketeers, whose support and stimulant have been our topmost source of alleviation throughout this journey. With gratefulness to Allah Almighty, we seek His guidance, wisdom, and blessings in all our trials. We extend our sincere thanks to our parents and Project Supervisor, whose constant support and stimulant drive us to achieve excellence. Our sincere appreciation also goes to our preceptors, whose mentorship and provocation have been vital in shaping our trip. This fidelity embodies our commitment to using technology for the betterment of society.

Acknowledgment

All praises and gratefulness to Almighty Allah, whose innumerable blessings enabled us to complete this design successfully. We extend our sincere thanks to Sir Almaas for opting our design and constantly inspiring us to strive for excellence. His precious guidance, perceptive suggestions, and unwavering stimulant have been necessary throughout this trip. We are equally grateful to our faculty members for their continuous encouragement and for imparting the knowledge and skills that shaped our academic journey. Special thanks to our university department for providing the necessary resources and support that enabled us to work on this project effectively. We also acknowledge the invaluable contribution of agricultural experts and farmers, whose insights into real-world challenges inspired the concept of this project. Their feedback helped us better understand the needs of the end-users and tailor the solution accordingly.

Table of Contents

CHAPTER 1	1
INTRODUCTION TO THE PROBLEM	1
1. Introduction to Problem	2
1.1 Executive Summary	2
1.2 Competitor's Analysis	2
1.2.2 PlantVillage Nuru App	2
1.2.2 CropIn Smart Farm	3
1.2.3 Why Our Application is Better Than Competitors	3
1.3 Purpose	3
1.4 Objectives	4
1.5 Existing Solution	4
1.6 Proposed Solution	5
Chapter 2	6
Software Requirement Specification	6
2.1 Introduction	7
2.1.1 Purpose	7
2.1.2 Scope	7
2.1.3 Overview	8
2.2 Overall Description	8
2.2.1 Product Perspective	8
2.2.1.1 Hardware Interface	8
2.2.1.2 Software Interface	8
2.2.1.3 Memory Constraint	8
2.2.2 User Characteristics	9
2.2.3 Constraints	9
2.2.4 Apportioning of Requirements	9
2.3 Product Functions	9
2.3.1 User Function	9
Chapter 3	14
USE CASE ANALYSIS	14
3.1 User Register	15

3.2 User Login	16
3.3 User Forget Password	17
3.4 User Logout	18
3.5 User Connects to Database	19
3.6 User Upload image	20
3.7 Image processing	21
3.8 Visualize Results	22
3.9 Recommendation	23
3.10 Aggregated Use Case Diagram	24
CHAPTER 4.....	25
DESIGN	25
4.1 Architecture Diagram	26
4.2 ERD	27
4.2.1 Logical Schema against ERD Diagram	28
4.2.2 Data dictionary against ERD	29
4.2.2.1 User Table	29
4.2.2.2 Field Table	29
4.2.2.3 Image Table	30
4.2.2.4 Disease Record Table	30
4.2.2.5 Weather DataType	31
4.2.2.6 Recommendation Table	31
4.3 Data flow diagram	32
4.3.1 The Level 0	32
4.3.2 The Level 1	32
4.4 Class Diagram	33
4.5 Activity Diagram	34
4.5.1 User Login	34
4.5.2 Image Upload	35
4.5.3 Image Validation	36
4.5.4 Resize and Normalize	37
4.5.5 AI Model Analyze	38
4.5.6 Retrieve Local Weather Data	39
4.5.7 Disease and Severity Detection	40

4.5.8 Generate Treatment Recommendations	41
4.5.9 Display Result	42
4.5.10 Aggregated Activity Diagram	43
4.6 Sequence Diagram	44
4.6.1 User Interface.....	44
4.6.2 Backend.....	45
4.6.3 AI Model.....	46
4.6.4 Weather	47
4.6.5 Result	48
4.6.6 Aggregated Sequence Diagram.....	49
4.7 Component Diagram	50
4.8 Deployment Diagram	51
4.9 State Transition Diagram	52
4.10 Collaboration Diagram	53
4.10.1 User Authentication	53
4.10.2 Image Upload and Processing.....	54
4.10.3 Disease Identification.....	55
4.10.4 Disease Management	56
4.10.5 Predictive Analytic	57
4.10.6 Alert and Notification.....	58
4.10.7 Aggregated Collaboration Diagram.....	59
CHAPTER 5.....	60
Testing.....	60
5.1 Test Case Specifications	61
5.1.1 User Register	61
5.1.2 User Login	63
5.1.3 User Forget Password	65
5.1.4 User Logout.....	67
5.1.5 User Connects to Database	69
5.1.6 User Upload image.....	71
5.1.7 Image processing	73
5.1.8 Visualize Results	75
5.1.9 Recommendation.....	77

5.2 Black Box Test Cases	79
5.2.1 Equivalence Partitions.....	79
5.2.1.1 User register.....	79
5.2.1.2 login.....	79
5.2.1.3 Forget password.....	79
5.2.1.4 User logout.....	80
5.2.1.5 Connect to database	80
5.2.1.6 Upload image	80
5.2.1.7 Image process	81
5.2.1.8 Visualize results.....	81
5.2.1.9 Recommendation.....	81
5.2.2 Boundary Value Analysis.....	82
5.2.2.1 User Registration.....	82
5.2.2.2 User Login	83
5.2.2.3 Forget Password.....	83
5.2.2.4 User Logout	84
5.2.2.5 Connect Database	84
5.2.2.6 Image Upload	85
5.2.2.7 Image Processing	85
5.2.2.8 Visualize Results.....	86
5.2.2.9 Recommendation.....	86
5.2.3 Decision Table Testing	87
5.2.3.1 User Register	87
5.2.3.2 Login	88
5.2.3.3 Forget Password.....	89
5.2.3.4 Logout	89
5.2.3.5 Connect to Database.....	90
5.2.3.6 Upload Image	91
5.2.3.7 Image Processing	92
5.2.3.8 Visualize Results.....	93
5.2.3.9 Recommendation.....	94
5.2.4 State transition Testing	95
5.2.4.1 User Register	95

5.2.4.2 User Login	95
5.2.4.3 User Forget Password	96
5.2.4.4 User Logout	96
5.2.4.5 User Connects to Database	96
5.2.4.6 User Upload image	97
5.2.4.7 Image processing	97
5.2.4.8 Visualize Results	97
5.2.4.9 Recommendation	97
5.2.5 Use Case Testing	98
5.2.5.1 User Register	98
5.2.5.2 User Login	98
5.2.5.3 User Forget Password	98
5.2.5.4 User Logout	99
5.2.5.5 User Connects to Database	99
5.2.5.6 User Upload image	99
5.2.5.7 Image processing	100
5.2.5.8 Visualize Results	100
5.2.5.9 Recommendation	101
5.3 White Box Test Cases	101
5.3.1 Cyclometric complexity	101
5.3.1.1 User Register	101
5.3.1.2 User Login	102
5.3.1.3 Forget Password	102
5.3.1.4 User Logout	102
5.3.1.5 User Connects to Database	102
5.3.1.6 Upload Image	103
5.3.1.7 Image Processing	103
5.3.1.8 Visualize Results	103
5.3.1.9 Recommendation	103
5.4 Performance testing	104
5.4.1 User Registration	104
5.4.2 User Login	104
5.4.3 Forget Password	104

5.4.4 User Logout	104
5.4.5 Connects to Database	105
5.4.6 Upload Image	105
5.4.7 Image Processing	105
5.4.8 Visualize Results	105
5.4.9 Recommendation System	105
5.5 Stress Testing	106
5.5.1 User Registration.....	106
5.5.2 User Login	106
5.5.3 Forget Password.....	106
5.5.4 User Logout	106
5.5.5 Connects to Database	106
5.5.6 Upload Image	107
5.5.7 Image Processing	107
5.5.8 Visualize Results	107
5.5.9 Recommendation System	107
5.6 System Testing	108
5.6.1 User Registration.....	108
5.6.2 User Login	108
5.6.3 Forget Password.....	108
5.6.4 User Logout	109
5.6.5 Connects to Database	109
5.6.6 Upload Image	109
5.6.7 Image Processing	110
5.6.8 Visualize Results	110
5.6.9 Recommendation.....	110
5.7 Regression Testing	111
5.7.1User Registration	111
5.7.2 User Login	111
5.7.3 Forget Password.....	112
5.7.4 User Logout	112
5.7.5 Connect to Database.....	112
5.7.6 Upload Image	113

5.7.7 Image Processing	113
5.7.8 Visualize Results	114
5.7.9 Recommendation	114
CHAPTER 6	115
Tool and Techniques	115
6.1 Applications and Tools	116
6.1.1 Visual Studio Code (VSCode)	116
6.1.2 Visual Paradigm	116
6.1.3 Google Cloab	116
6.2 Techniques	116
6.2.1 Machine Learning Models	116
6.2.2 React.js	117
6.2.3 JavaScript (React.js)	117
6.2.4 AOS (Animate on Scroll)	117
6.2.5 React Icons	117
6.2.6 Python	117
6.2.7 Machine Learning Integration	117
6.3 Framework	118
6.3.1 Django	118
6.3.2 Engine TensorFlow	118
CHAPTER 7	119
Summary and Conclusion	119
7.1 Summary	120
7.2 Conclusion	121
CHAPTER 8	122
User Manual	122
8.1 User Register	123
8.2 User Login	124
8.2 User Forget Password	125
8.2 Dashboard Overview	126
8.3 Uploading Images	127
8.4 Analyzing Images	128
8.5 Viewing Results	129

8.6 Recommendations	130
8.7 About Us.....	131
8.8 Contact Us.....	131
References.....	132

List of Figures

Figure 3.1: User Register	15
Figure 3.2: User Login	16
Figure 3.3: User Forget Password	17
Figure 3.4: User Logout	18
Figure 3.5: User Connects to Database.....	19
Figure 3.6: User Upload image.....	20
Figure 3.7: Image processing	21
Figure 3.8: Visualize Results.....	22
Figure 3.9: Recommendation.....	23
Figure 3.10: <i>Aggregated Use Case Diagram</i>	24
Figure 4.11 Architecture	26
Figure 4.12 ERD	27
Figure 4.13 Level 0	32
Figure 4.14 Level 1	32
Figure 4.15 Class Diagram	33
Figure 4.16 Activity Diagram User Login	34
Figure 4.17 Activity Diagram Image Upload	35
Figure 4.18 Activity Diagram Image Validation.....	36
Figure 4.19 Activity Diagram Resize and Normalize	37
Figure 4.20 Activity Diagram AI Model Analyze.....	38
Figure 4.21 Activity Diagram Retrieve Local Weather Data.....	39
Figure 4.22 Activity Diagram Disease and Severity Detection	40
Figure 4.23 Activity Diagram Generate Treatment Recommendations	41
Figure 4.24 Activity Diagram Display Result	42
Figure 4.25 Aggregated Activity Diagram	43
Figure 4.26 Sequence Diagram User Interface	44
Figure 4.27 Sequence Diagram Backend.....	45
Figure 4.28 Sequence Diagram AI Model.....	46

Figure 4.29 Sequence Diagram Weather	47
Figure 4.30 Sequence Diagram Result	48
Figure 4.31 Sequence Diagram Aggregated Sequence Diagram.....	49
Figure 4.32 Component Diagram.....	50
Figure 4.33 Deployment Diagram	51
Figure 4.34 State Transition Diagram	52
Figure 4.35 Collaboration Diagram User Authentication.....	53
Figure 4.36 Collaboration Diagram Image Upload and Processing.....	54
Figure 4.37 Collaboration Diagram Disease Identification	55
Figure 4.38 Collaboration Diagram Disease Management	56
Figure 4.39 Collaboration Diagram Predictive Analytics.....	57
Figure 4.40 Collaboration Diagram Alert and Notification	58
Figure 4.41 Aggregated Collaboration Diagram.....	59
Figure 8.42 User Register	123
Figure 8.43 User Login	124
Figure 8.44 User Forget Password	125
Figure 8.45 Dashboard Overview.....	126
Figure 8.46 Dashboard Overview.....	127
Figure 8.47 Analyzing Images.....	128
Figure 8.48 Viewing Results	129
Figure 8.49 Recommendations	130
Figure 8.50 About Us.....	131
Figure 8.51 Contact Us.....	131

List of Tables

Table 2.1: <i>User Registration and Authentication</i>	10
Table 2.2: <i>Image Upload and Preprocessing</i>	10
Table 2.3: <i>Disease Identification</i>	11
Table 2.4: <i>Severity Assessment</i>	11
Table 2.5: <i>Disease Management Recommendations</i>	12
Table 2.6: <i>Predictive Analytics</i>	12
Table 2.7: <i>Real-Time Data Collection</i>	13
Table 2.8: <i>Alerts and Notifications</i>	13
Table 3.9: Use Case # 1, User Registration	15
Table 3.10: Use Case # 2, User Login Account.....	16
Table 3.11: Use Case # 3, User Forget Password	17
Table 3.12: Use Case # 4, User Logout	18
Table 3.13: Use Case # 5, User Connects to Database	19
Table 3.14: <i>Use Case # 6, User Upload image</i>	20
Table 3.15: <i>Use Case # 7, Image processing</i>	21
Table 3.16: <i>Use Case # 8, Visualize Results</i>	22
Table 3.17: <i>Use Case # 9, Recommendation</i>	23
Table 4.18: User Table.....	29
Table 4.19: Field Table	29
Table 4.20: Image Table	30
Table 4.21: Disease Record Table	30
Table 4.22: DataType.....	31
Table 4.23: Recommendation Table	31
Table 5.24: Test Case Specification signup successful.....	61
Table 5.25: Test Case Specification signup failure.....	62
Table 5.26: Test Case Specification login successful	63
Table 5.27: Test Case Specification login failure	64
Table 5.28: Test Case Specification forget password successful.....	65

Table 5.29: Test Case Specification forget password failure.....	66
Table 5.30: Test Case Specification logout successful	67
Table 5.31: Test Case Specification logout failure	68
Table 5.32: Test Case Specification Connects to Database successful.....	69
Table 5.33: Test Case Specification Connects to Database failure.....	70
Table 5.34: Test Case Specification Upload image successful.....	71
Table 5.35: Test Case Specification Upload image failure	72
Table 5.36: Test Case Specification Image processing successful	73
Table 5.37: Test Case Specification Image processing failure	74
Table 5.38: Test Case Specification Visualize Results successful.....	75
Table 5.39: Test Case Specification Visualize Results failure.....	76
Table 5.40: Test Case Specification Recommendation successful	77
Table 5.41: Test Case Specification Recommendation failure	78
Table 5.42: Equivalence Partitions User register	79
Table 5.43: Equivalence Partitions Login.....	79
Table 5.44: Equivalence Partitions Forget password.....	79
Table 5.45: Equivalence Partitions User logout.....	80
Table 5.46: Equivalence Partitions Connect to database	80
Table 5.47: Equivalence Partitions Upload image.....	80
Table 5.48: Equivalence Partitions Image process	81
Table 5.49: Equivalence Partitions Visualize results.....	81
Table 5.50: Equivalence Partitions Recommendation	81
Table 5.51: Boundary Value Analysis User Registration	82
Table 5.52: Boundary Value Analysis User Login	83
Table 5.53: Boundary Value Analysis Forget Password	83
Table 5.54: Boundary Value Analysis User Logout	84
Table 5.55: Boundary Value Analysis Connect Database	84
Table 5.56: Boundary Value Analysis Image Upload	85
Table 5.57: Boundary Value Analysis Image Processing.....	85
Table 5.58: Boundary Value Analysis Visualize Results	86

Table 5.59: Boundary Value Analysis Recommendation	86
Table 5.60: Decision Table Testing User Register	87
Table 5.61: Decision Table Testing Login.....	88
Table 5.62: Decision Table Testing Forget Password	89
Table 5.63: Decision Table Testing Logout.....	89
Table 5.64: Decision Table Testing Connect to Database	90
Table 5.65: Decision Table Testing Upload Image	91
Table 5.66: Decision Table Testing Image Processing	92
Table 5.67: Decision Table Testing Visualize Results	93
Table 5.68: Decision Table Testing Recommendation	94
Table 5.69: State transition Testing User Register.....	95
Table 5.70: State transition Testing User Login	95
Table 5.71: State transition Testing User Forget Password	96
Table 5.72: State transition Testing User Logout	96
Table 5.73: State transition Testing Connects to Database	96
Table 5.74: State transition Testing Upload image	97
Table 5.75: State transition Testing Image processing	97
Table 5.76: State transition Testing Visualize Results.....	97
Table 5.77: State transition Testing Recommendation	97
Table 5.78: Use Case Testing User Register.....	98
Table 5.79: Use Case Testing User Login	98
Table 5.80: Use Case Testing User Forget Password	98
Table 5.81: Use Case Testing User Logout	99
Table 5.82: Use Case Testing User Connects to Database	99
Table 5.83: Use Case Testing User Upload image	99
Table 5.84: Use Case Testing Image processing	100
Table 5.85: Use Case Testing Visualize Results.....	100
Table 5.86: Use Case Testing Recommendation	101

Abstract

Wheat leaf diseases, such as rust, powdery mildew, and leaf blight, pose significant threats to global wheat production by reducing crop yield and quality. Early and accurate detection is critical to managing these diseases effectively, but traditional manual methods are time-consuming, error-prone, and often inaccessible to small-scale farmers. This project proposes an AI-driven wheat leaf disease detection system that utilizes machine learning and image processing techniques to identify and classify diseases from leaf images. The system provides a user-friendly interface for farmers and agricultural researchers to upload images via web applications. It delivers instant diagnostic results along with treatment recommendations, enabling timely interventions. By offering a cost-effective, scalable, and efficient solution, this system aims to reduce crop losses and promote sustainable agricultural practices.

CHAPTER 1

INTRODUCTION TO THE PROBLEM

1. Introduction to Problem

Wheat leaf diseases, specifically yellow rust, brown rust, and black rust is a type of fungal diseases which pose a major threat to global wheat production in reducing growth and quality. These diseases spread rapidly under favorable conditions making timely detection and involvement is crucial. However, traditional methods for identifying these diseases are often slow and labor intensive also early detection and management are crucial to mitigating these effects. This project aims to develop a technology driven solution for the early detection and management of these diseases to support sustainable wheat production.

1.1 Executive Summary

This project addresses the critical issue of wheat leaf fungal diseases focusing on three major types: yellow rust, brown rust, and black rust. These diseases are among the most destructive to wheat crops globally causing significant growth losses and threatening food security. Traditional detection methods are often slow requiring manual inspection and specialized knowledge which delays timely intervention. The proposed solution involves developing an automated system for the early detection and classification of these diseases using machine learning and image processing. The goal is to offer a user friendly, efficient and scalable tool that can be deployed across different wheat-growing regions to support farmers in disease management. Then we will create the website and adjust our model into this website which helps non-technical user to use the model easily.

1.2 Competitor's Analysis

Competitor analysis involves evaluating existing solutions in the market:

1.2.2 PlantVillage Nuru App

- **Features:** PlantVillage, developed in collaboration with Penn State University and the United Nations, includes an app named Nuru that uses artificial intelligence to detect plant diseases. The app allows users to upload images of plants, which are then analyzed for signs of diseases.
- **Strengths:** The app is widely accessible, free to use, and supports multiple crops.
- **Limitations:** Nuru's general focus on multiple crops may limit its accuracy for specific wheat rusts, as it does not provide highly specialized diagnostics for leaf rust, stripe rust, and stem rust.

1.2.2 CropIn Smart Farm

- **Features:** CropIn provides an AI-based solution that includes disease detection crop monitoring and yield prediction primarily aimed at large farms and agribusinesses. Smart Farm enables remote monitoring, with disease detection as part of a larger farm management suite.
- **Strengths:** It offers robust farm management capabilities, integrating disease detection with satellite data
- **Limitations:** The solution may be too complex and costly for individual farmers, especially those focused solely on wheat rust detection, as it is designed more for comprehensive farm management.

1.2.3 Why Our Application is Better Than Competitors

- **Highly Specialized:** Focuses specifically on detecting wheat rust diseases (yellow rust, brown rust, black rust) for improved accuracy, unlike generalist apps like Nuru.
- **Farmer-Friendly:** Designed for individual farmers with a simple interface—no technical expertise or training required.
- **Balanced & Practical:** Combines powerful AI detection with an easy-to-use, field-ready app, unlike complex systems like CropIn or research-based models.
- **Real-Time Results:** Provides instant disease detection through image capture, eliminating delays associated with lab testing or external processing.

1.3 Purpose

Wheat is one of the most essential staple crops worldwide, providing a critical source of food for millions of people. However, wheat production faces serious threats from fungal diseases particularly yellow rust, brown rust, and black rust which can lead to substantial yield losses and impact food security. The purpose of this project is to develop an efficient, cost-effective, and scalable solution to detect wheat leaf diseases using advanced AI techniques. This system will empower farmers and agricultural stakeholders to improve disease management practices and enhance wheat production sustainability.

1.4 Objectives

- **Develop an Accurate Disease Detection Model:** Create a machine learning model capable of accurately identifying yellow rust, brown rust, and black rust from images of wheat leaves.
- **Ensure High Classification Accuracy:** Achieve a minimum detection accuracy rate of 90% for each disease type to ensure reliable disease identification in real-world conditions.
- **Design a User-Friendly Interface:** Build an intuitive interface that allows users, including farmers and agricultural consultants, to upload leaf images and receive rapid disease diagnosis.
- **Timely Diagnosis:** Provide rapid disease diagnosis to enable quick decision-making and prompt disease management.
- **Responsive Design:** Ensure the website is fully responsive and works well on all devices, Efficient Data Upload and Processing: Provide a fast and reliable interface for users to upload images and data without delays.
- **Offer Tailored Treatment Recommendations:** Generate actionable treatment recommendations based on the detected disease type and severity to aid in effective disease management.
- **Facilitate Preventive Measures:** Include educational resources and preventive strategies to help users reduce the risk of future infections.
- **Validate System Effectiveness through Field Trials:** Conduct field trials to test the system's performance in real agricultural environments, refining the model based on real-world feedback.
- **Ensure Data Privacy and Security:** Implement robust data security measures to protect users' sensitive information and comply with relevant data protection regulations.
- **Develop Website:** Develop a user-friendly website that will help the farmers to detect diseases in the crops. The main objective of the model is to help the farmers prevent losses by taking precautionary measures that our model will provide in order to address the issue.

1.5 Existing Solution

Research in recent years has explored various approaches for identifying and managing these fungal diseases.

- **Traditional Manual Observation:** Traditional methods rely on visual inspection and expert identification which are effective but resource-intensive. Studies have investigated molecular techniques for pathogen detection but these methods are often inaccessible to farmers due to cost and technical requirements.
- **Commercial Tools and Applications:** Plant health monitoring apps exist but lack precision for specific diseases. Research institutions deploy models, but these are often not user-accessible.
- **Remote Sensing Technologies:** Satellite or drone-based tools are used but are costly and require high-end infrastructure.

1.6 Proposed Solution

The proposed solution is an AI-powered wheat leaf disease detection system that integrates:

- **Image Processing and Machine Learning:** Utilizing convolutional neural networks (CNNs) to classify diseases based on leaf images.
- **Web Interface:** Allowing farmers to upload leaf images and receive instant diagnostic results.
- **Localized Recommendations:** The system will provide treatment advice based on the detected disease and the user's geographical location.
- **Affordable Deployment:** Ensuring the tool is accessible to small-scale farmers through minimal hardware requirements.

Chapter 2

Software Requirement Specification

3.1 User Register

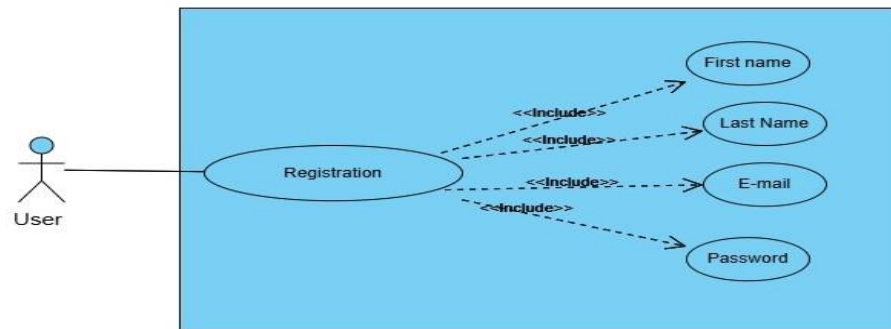


Figure 3.1: User Register

Table 3.9: Use Case # 1, User Registration

Use Case ID	UC_01	
Use Case Name	User Registration	
Description	The person will sign up as a user by giving his credentials.	
Primary Actor	user	
Secondary Actor	System	
Pre-Condition	The member will fill out the sign-up form	
Post-Condition	The person will be registered as a user.	
Basic Flow	Actor Action	System Action
	The actor will add the credentials of the member and accept it for registration	The system will check whether the credentials entered by the person are correct or not.
Alternate Flow	• Generate an error message for not fulfilling all the requirements. • Generate an error message for providing an invalid email id. • Generate an error message because the email is already taken/registered.	

3.2 User Login

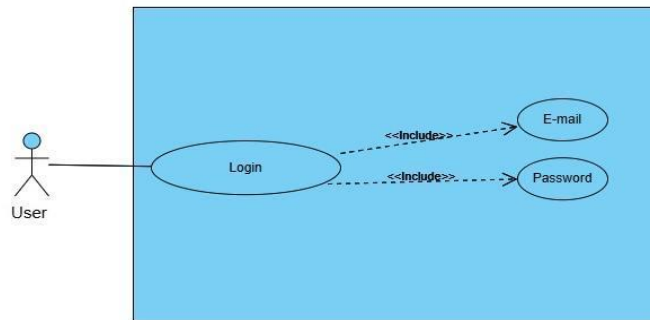


Figure 3.2: User Login

Table 3.10: Use Case # 2, User Login Account

Use Case ID	UC_02	
Use Case Name	user login account	
Description	The person will enter login details.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The member will enter the login details	
Post-Condition	The person will be login as a user.	
Basic Flow	Actor Action	System Action
	The actor will enter the email and password for a login account.	The system will check whether the login information is correct or not and log in to the user account.
Alternate Flow	• Generate an error message for providing an invalid emailid. • Generate an error message for providing an invalidpassword. • Generate an error message because u is not registered yourself before.	

3.3 User Forget Password

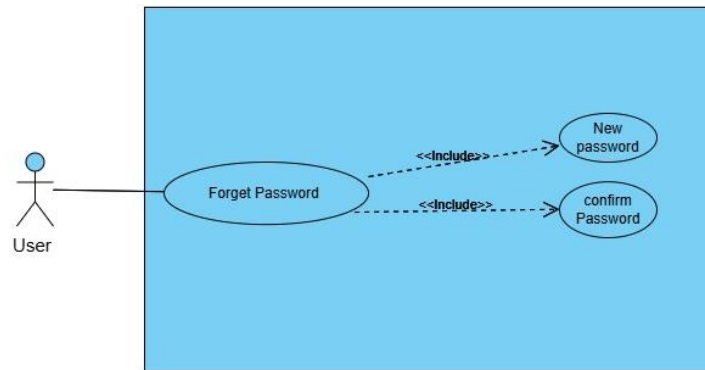


Figure 3.3: User Forget Password

Table 3.11: Use Case # 3, User Forget Password

Use Case ID	UC_03	
Use Case Name	User forget password	
Description	The person will forget the account password.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The member will enter the new password	
Post-Condition	The person will successfully reset the password.	
Basic Flow	Actor Action	System Action
	The actor will enter the new password that he/she wants to change.	The system will verify the password and reset the password successfully.
Alternate Flow	Cannot remember the previous password.	

3.4 User Logout

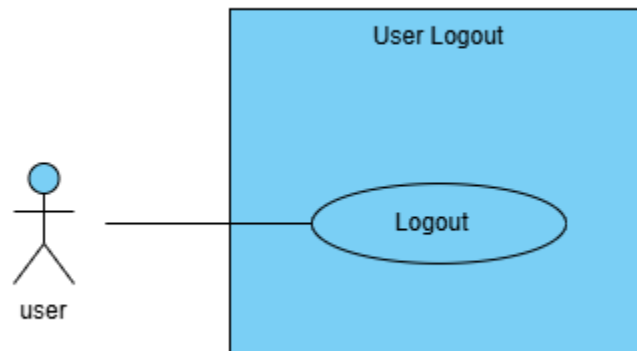


Figure 3.4: User Logout

Table 3.12: Use Case # 4, User Logout

Use Case ID	UC_04	
Use Case Name	User logout account	
Description	The person will log out of the account.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The member will press the logout button.	
Post-Condition	The account will be successfully logout.	
Basic Flow	Actor Action	System Action
	The actor will press the logout button to logout the account	The system will log out of the account successfully.
Alternate Flow	None	

3.5 User Connects to Database

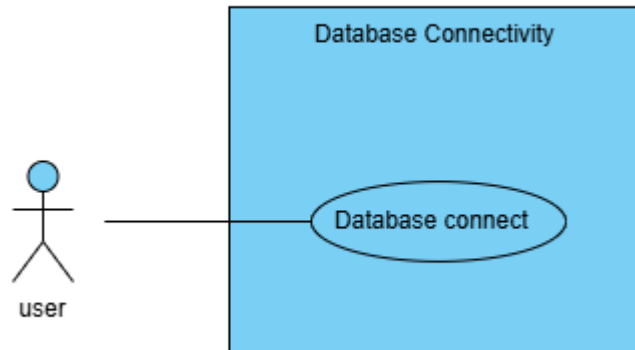


Figure 3.5: User Connects to Database

Table 3.13: Use Case # 5, User Connects to Database

Use Case ID	UC_05	
Use Case Name	User Connects to Database	
Description	The user initiates the process of connecting to a database by providing necessary connection details.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The user is logged into the system.	
Post-Condition	The user's database connection is established successfully, or an error message is provided. .	
Basic Flow	Actor Action	System Action
	Actor navigates, enters database details, initiates connection test, and acknowledges success or not.	The System presents interface, validates details, establishes connection, securely saves, and acknowledges success or not.
Alternate Flow	If the selected file format or content is invalid, the system notifies the user, and the process is halted.	

3.6 User Upload image

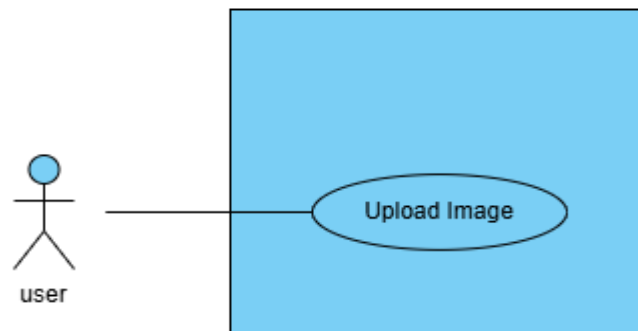


Figure 3.6: User Upload image

Table 3.14: Use Case # 6, User Upload image

Use Case ID	UC_06	
Use Case Name	User Upload image	
Description	The user uploads images for analysis.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The user is logged into the system.	
Post-Condition	The image is successfully uploaded for analysis, or an error message is provided.	
Basic Flow	Actor Action	System Action
	The user upload image.	The system analyzed the image and detect the disease. If analysis is successful, the system detect the image and upload the results to the user.
Alternate Flow	● Generate the error if the image is corrupted. ● Generate the error if the image is irrelevant.	

3.7 Image processing

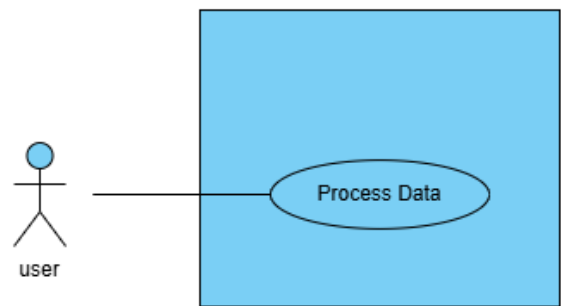


Figure 3.7: Image processing

Table 3.15: Use Case # 7, Image processing

Use Case ID	UC_07	
Use Case Name	Image processing	
Description	The image that is being uploaded is processed to detect weather the wheat leaf is diseased or healthy.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The user is logged into the system and has uploaded image.	
Post-Condition	The system processes the image and tell the result or an error message is provided.	
Basic Flow	Actor Action	System Action
	The user navigates to the "Prompt Data Analysis" section and inputs a natural language query.	The system processed the image and detect the disease. If processing is successful, the system detect the image and tell either the leaf is diseased or healthy.
Alternate Flow	● Generate the error if the image is corrupted. ● Generate the error if the image is irrelevant.	

3.8 Visualize Results

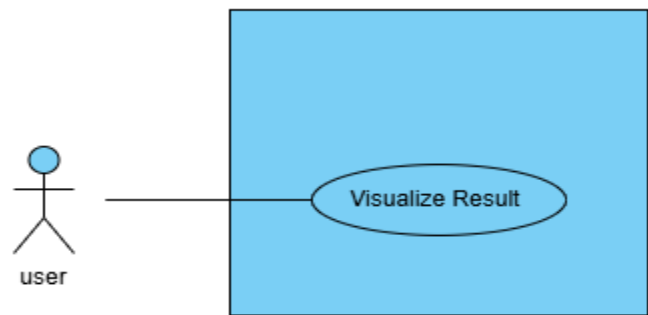


Figure 3.8: Visualize Results

Table 3.16: Use Case # 8, Visualize Results

Use Case ID	UC_08	
Use Case Name	Visualize Results	
Description	The user visualizes the results of a image.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	The user has completed a image analysis task and wants to visualize the results.	
Post-Condition	Representations of results are successfully displayed.	
Basic Flow	Actor Action	System Action
	The actor chooses the "Visualize" option.	The system processes the request, generates visual representations, and displays them for user interaction.
Alternate Flow	None	

3.9 Recommendation

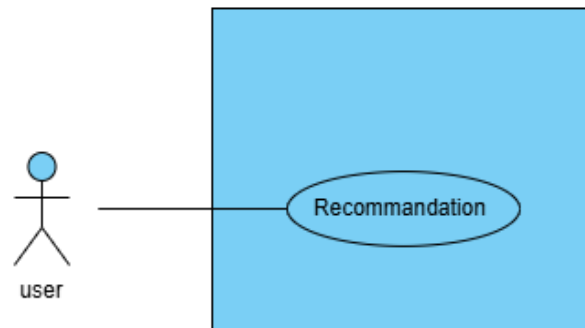


Figure 3.9: Recommendation

Table 3.17: *Use Case # 9, Recommendation*

Use Case ID	UC_09	
Use Case Name	Recommendation	
Description	The system give the recommendation about the disease.	
Primary Actor	User	
Secondary Actor	System	
Pre-Condition	Representations of results are successfully displayed.	
Post-Condition	The system gives the solution of problem.	
Basic Flow	Actor Action	System Action
	The actor chooses the "recommendation" option.	The system gives the recommendation about the disease if any disease is occurred.
Alternate Flow	None	

3.10 Aggregated Use Case Diagram

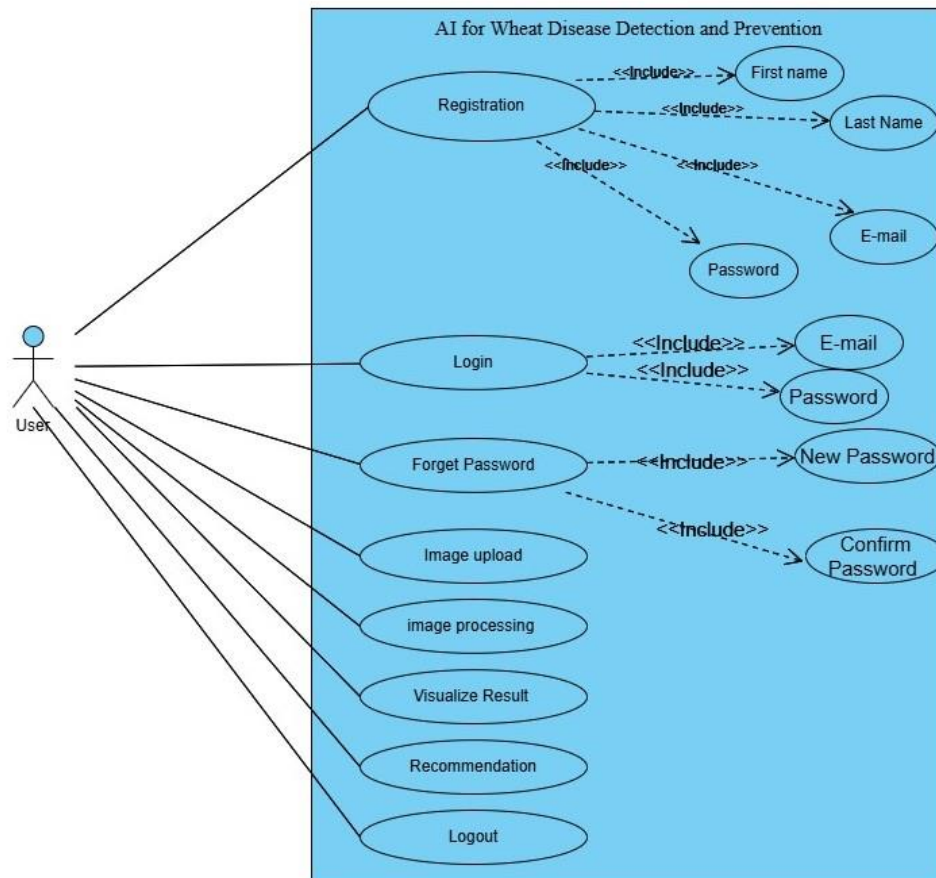


Figure 3.10: *Aggregated Use Case Diagram*

CHAPTER 4

DESIGN

4.1 Architecture Diagram

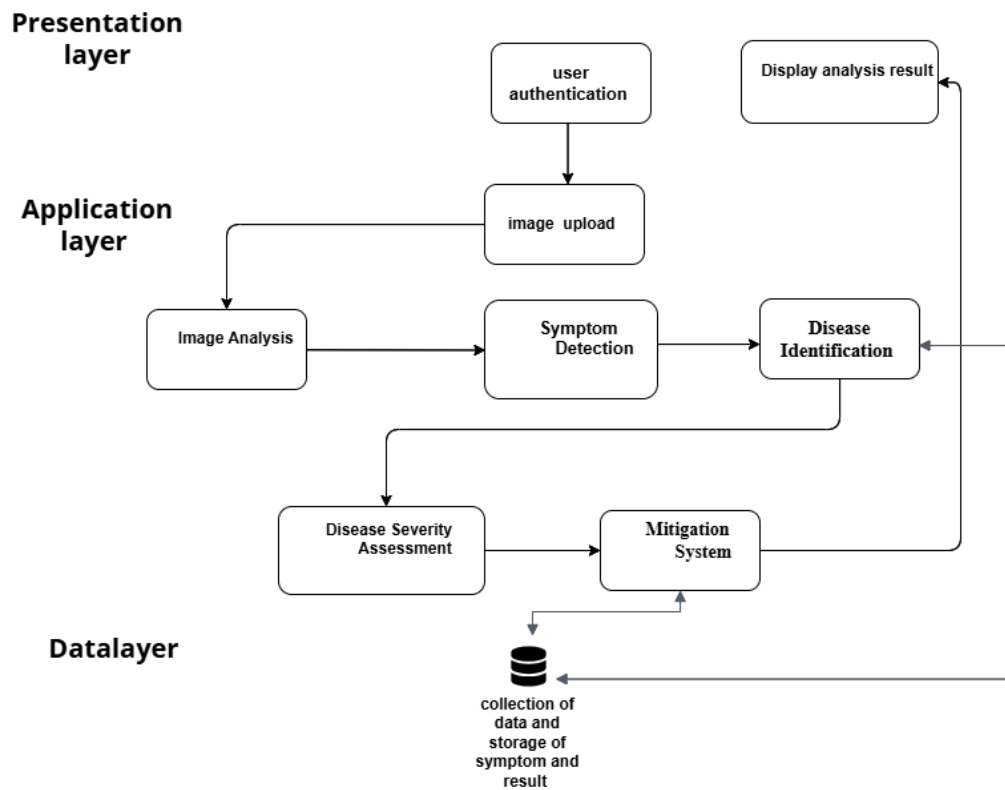


Figure 4.11 Architecture

4.2 ERD

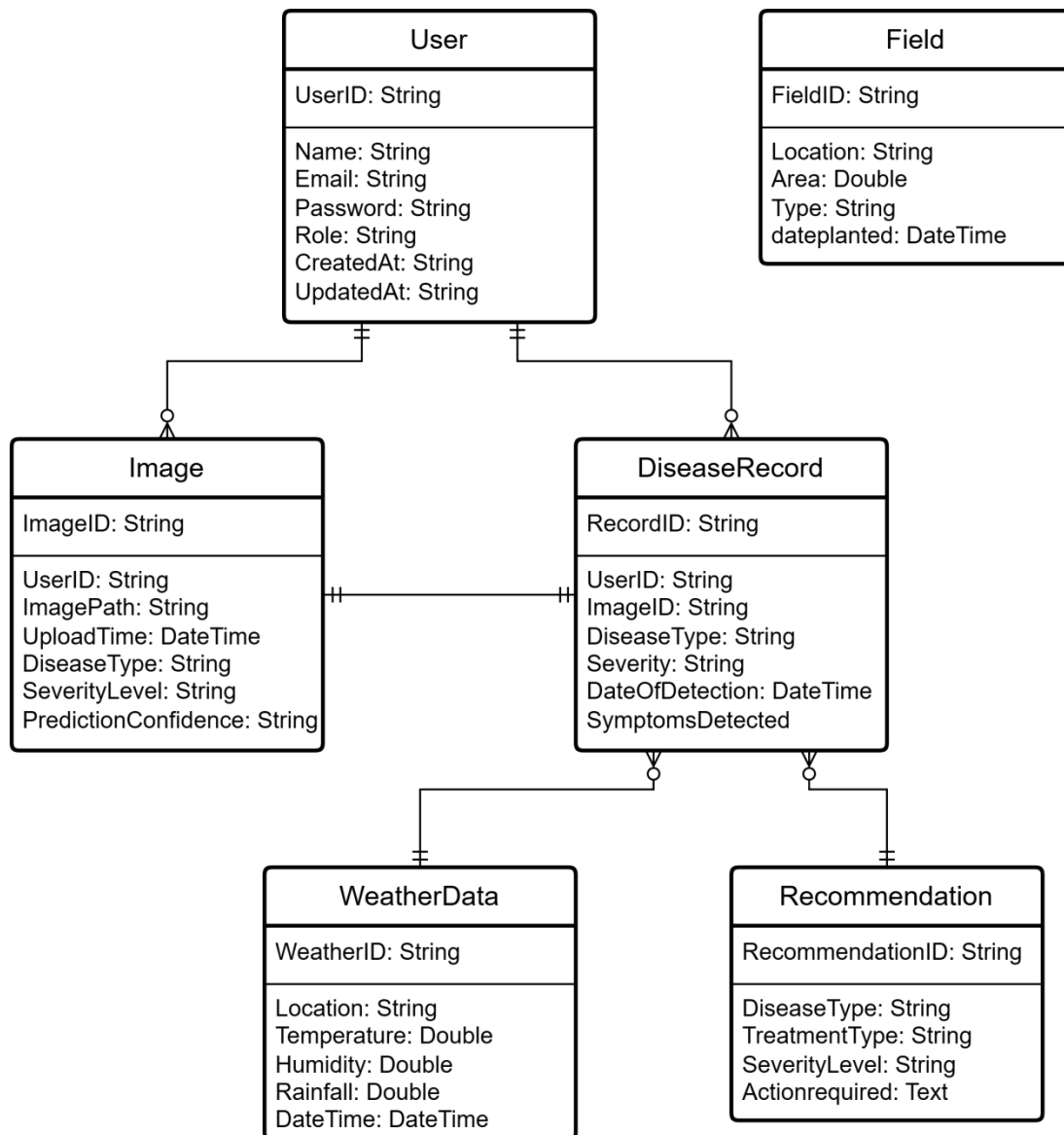


Figure 4.12 ERD

4.2.1 Logical Schema against ERD Diagram

Each entity in the ERD corresponds to a table, and the relationships (represented by lines) show how tables connect through foreign keys.

Here's the mapping of entities to tables:

User Table

Primary Key: UserID

Attributes: Name, Email, Password, Role, CreatedAt, UpdatedAt

Field Table

Primary Key: FieldID

Attributes: Location, Area, Type, DatePlanted

Image Table

Primary Key: ImageID

Foreign Key: UserID (References User Table)

Attributes: ImagePath, UploadTime, DiseaseType, SeverityLevel, PredictionConfidence

DiseaseRecord Table

Primary Key: RecordID

Foreign Keys: UserID (References User Table), ImageID (References Image Table)

Attributes: DiseaseType, Severity, DateOfDetection, SymptomsDetected

WeatherData Table

Primary Key: WeatherID

Attributes: Location, Temperature, Humidity, Rainfall, DateTime

Recommendation Table

Primary Key: RecommendationID

Attributes: DiseaseType, TreatmentType, SeverityLevel, ActionRequired

4.2.2 Data dictionary against ERD

4.2.2.1 User Table

Table 4.18: User Table

Column Name	Data Type	Primary Key	Foreign Key	Description
UserID	String	Yes		Unique identifier for user
Name	String	No		User's full name
Email	String	No		User's email address
Password	String	No		User's password
Role	String	No		Role of the user
CreatedAt	String	No		Account creation date
UpdateAt	String	No		Last update date

4.2.2.2 Field Table

Table 4.19: Field Table

Column Type	Data Type	Primary Key	Foreign Key	Description
FieldId	String	Yes		Unique identifier for field
Location	String	No		Location of the field
Area	Double	No		Area of the field in acres/hectares
Tpye	String	No		Type of field (e.g., crop type)
Date Planted	Datetime	No		Date when planting occurred

4.2.2.3 Image Table

Table 4.20: Image Table

Column Name	Data Type	Primary Key	Foreign Key	Description
ImageId	String	Yes		Unique identifier for image
UserId	String	No	References User(UserId)	User who uploaded the image
Image Path	String	No		Path where the image is stored
Upload Time	DateTime	No		Timestamp of image upload
Disease Type	String	No		Predicted disease type
Severity Level	String	No		Severity level of the disease
Prediction Confidence	String	No		Confidence level of prediction

4.2.2.4 Disease Record Table

Table 4.21: Disease Record Table

Column Table	Data Type	Primary Key	Foreign Key	Description
RecordId	String	Yes		Unique identifier for the record
UserId	String	No	References User(UserID)	User associated with the record
ImageId	String	No	References Image(ImageID)	Image associated with the record
Disease Type	String	No		Type of detected disease
Severity	String	No		Severity level of the disease
Data Of Detection	DateTime	No		Date when disease was detected
Symptoms Detected	String	No		Symptoms observed

4.2.2.5 Weather Data Type

Table 4.22: Data Type

Column Name	Data Type	Primary Key	Foreign Key	Description
WeatherId	String	Yes		Unique identifier for weather record
Location	String	No		Location of weather data
Temperature	Double	No		Recorded temperature
Humidity	Double	No		Humidity level
Rainfall	Double	No		Rainfall amount
Datetime	Date Time	No		Date and time of the record

4.2.2.6 Recommendation Table

Table 4.23: Recommendation Table

Column Name	Data Type	Primary Key	Foreign Key	Description
RecommendationID	String	Yes		Unique identifier for recommendation
Disease Type	String	No		Disease type being treated
Treatment Type	String	No		Type of treatment
Severity Level	String	No		Severity level of the disease
Action Required	Text	No		Recommended actions

4.3 Data flow diagram

4.3.1 The Level 0

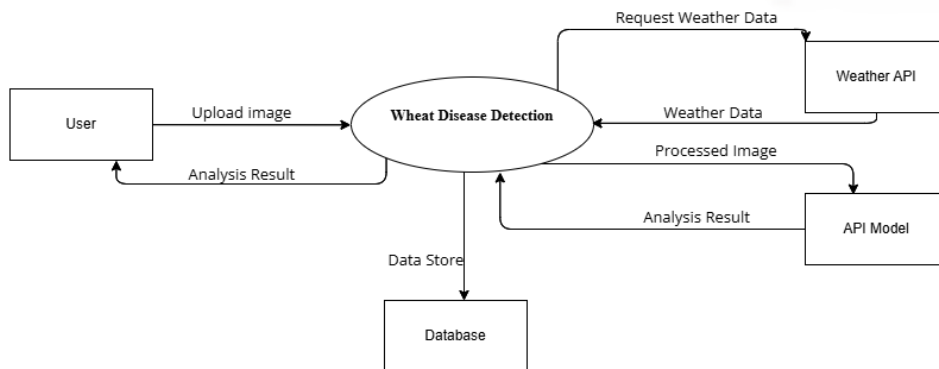


Figure 4.13 Level 0

4.3.2 The Level 1

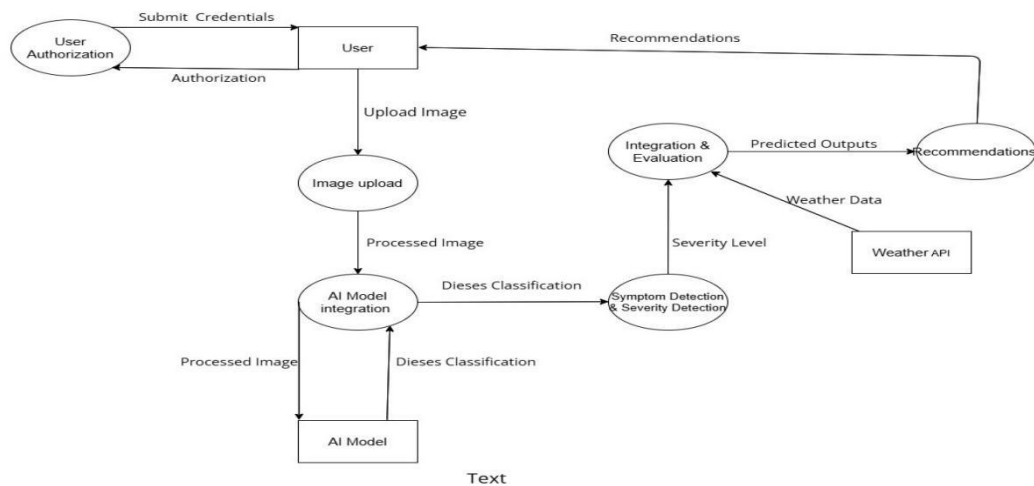


Figure 4.14 Level 1

4.4 Class Diagram

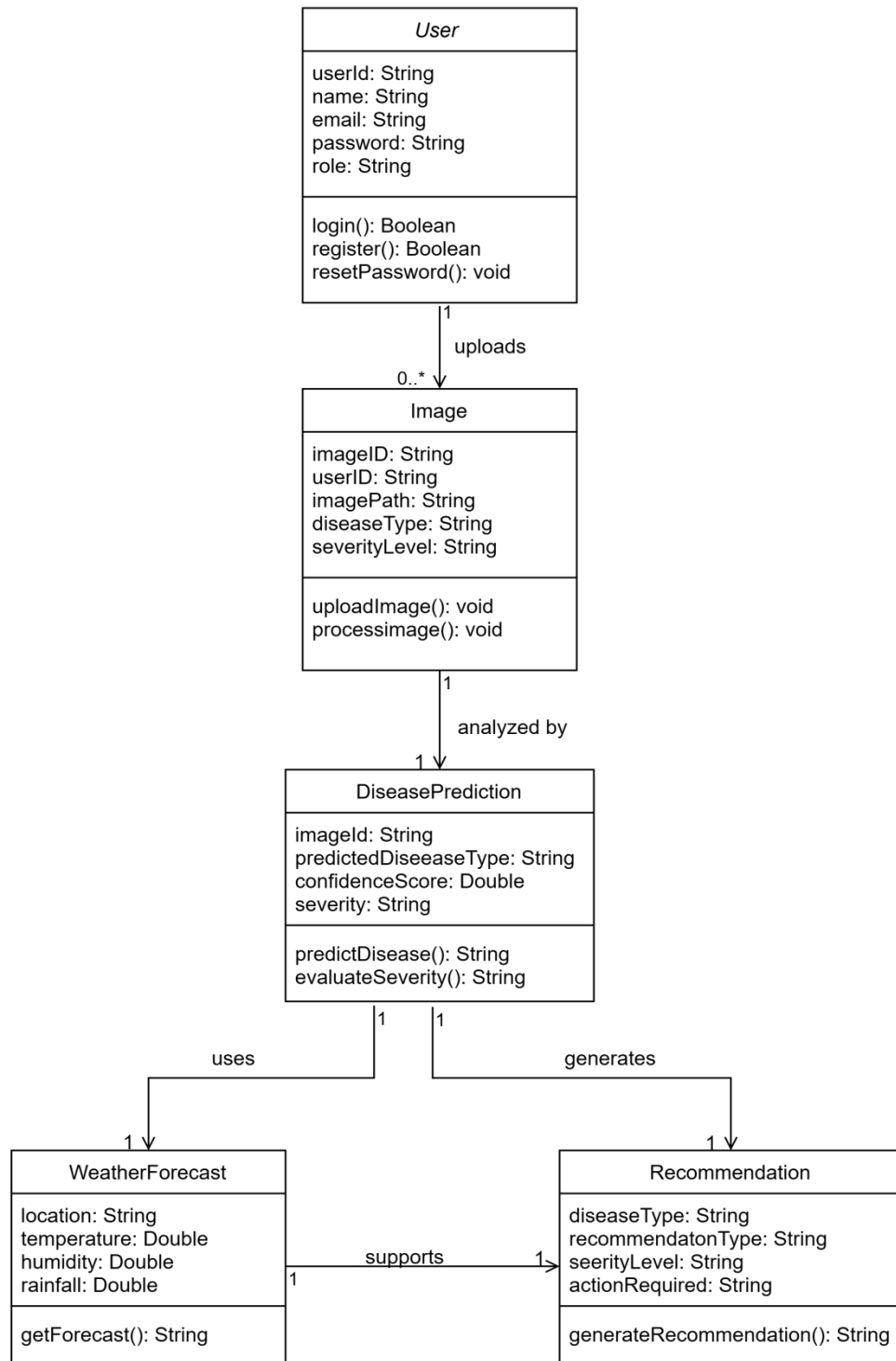


Figure 4.15 Class Diagram

4.5 Activity Diagram

4.5.1 User Login

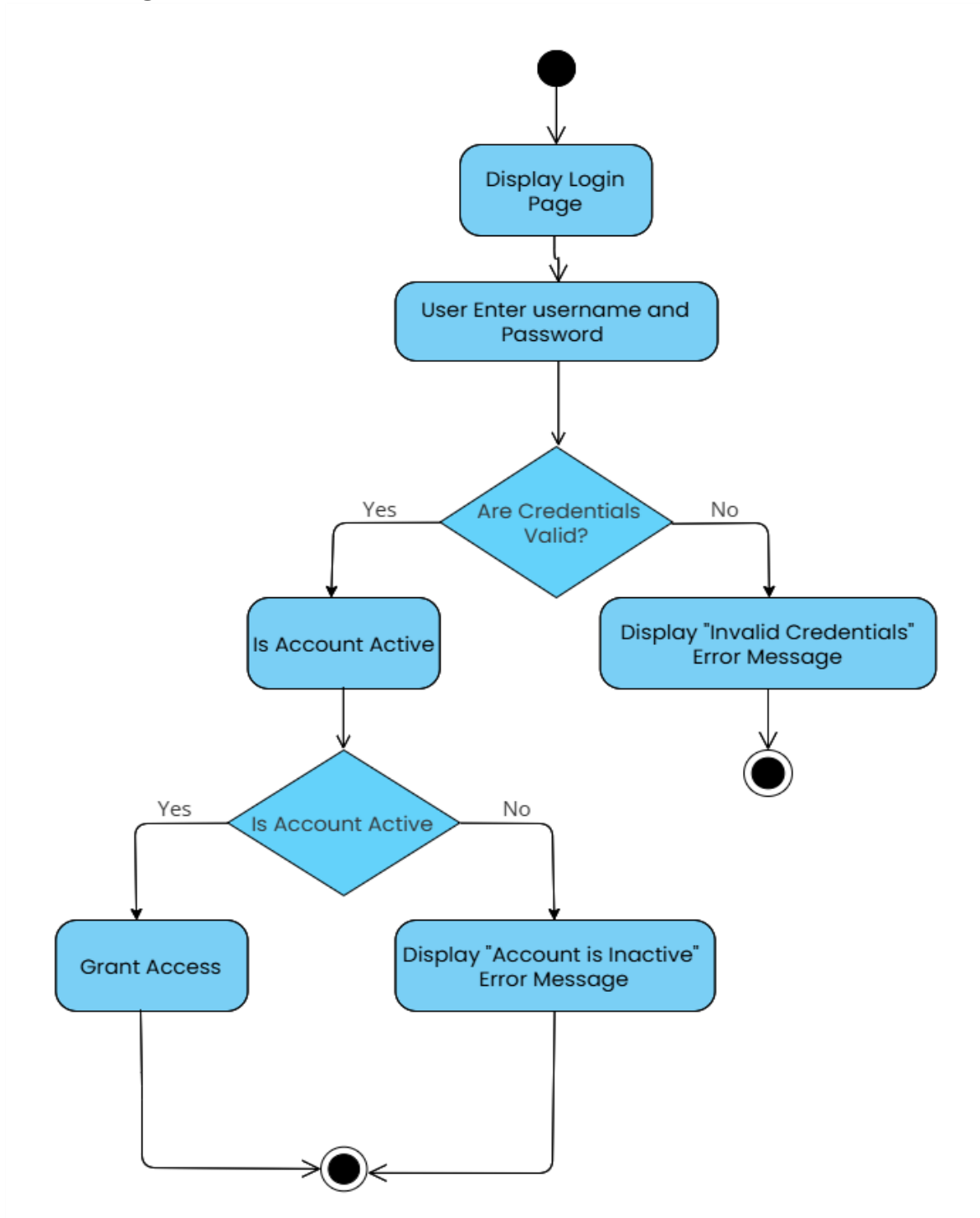


Figure 4.16 Activity Diagram User Login

4.5.2 Image Upload

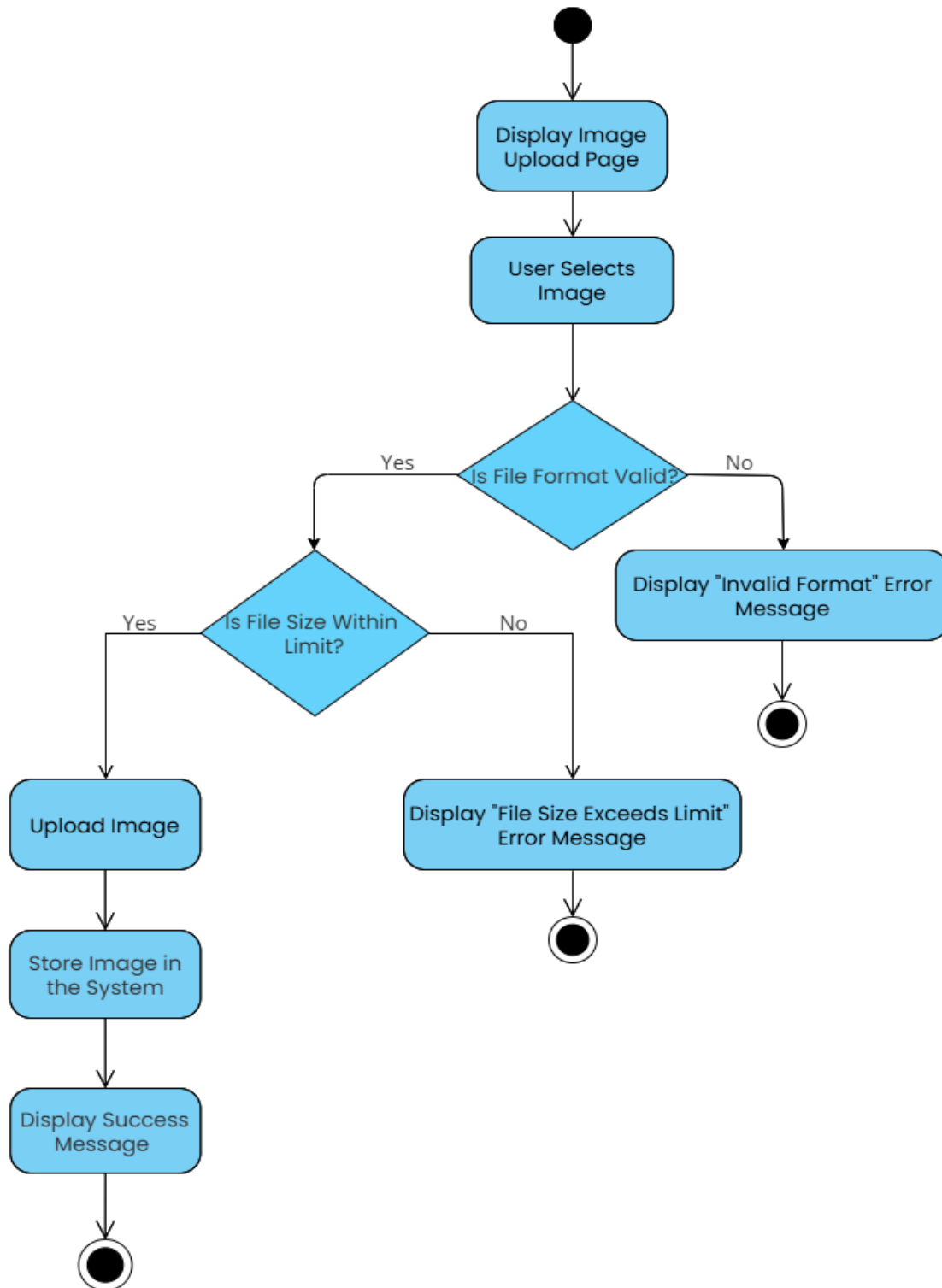


Figure 4.17 Activity Diagram Image Upload

4.5.3 Image Validation

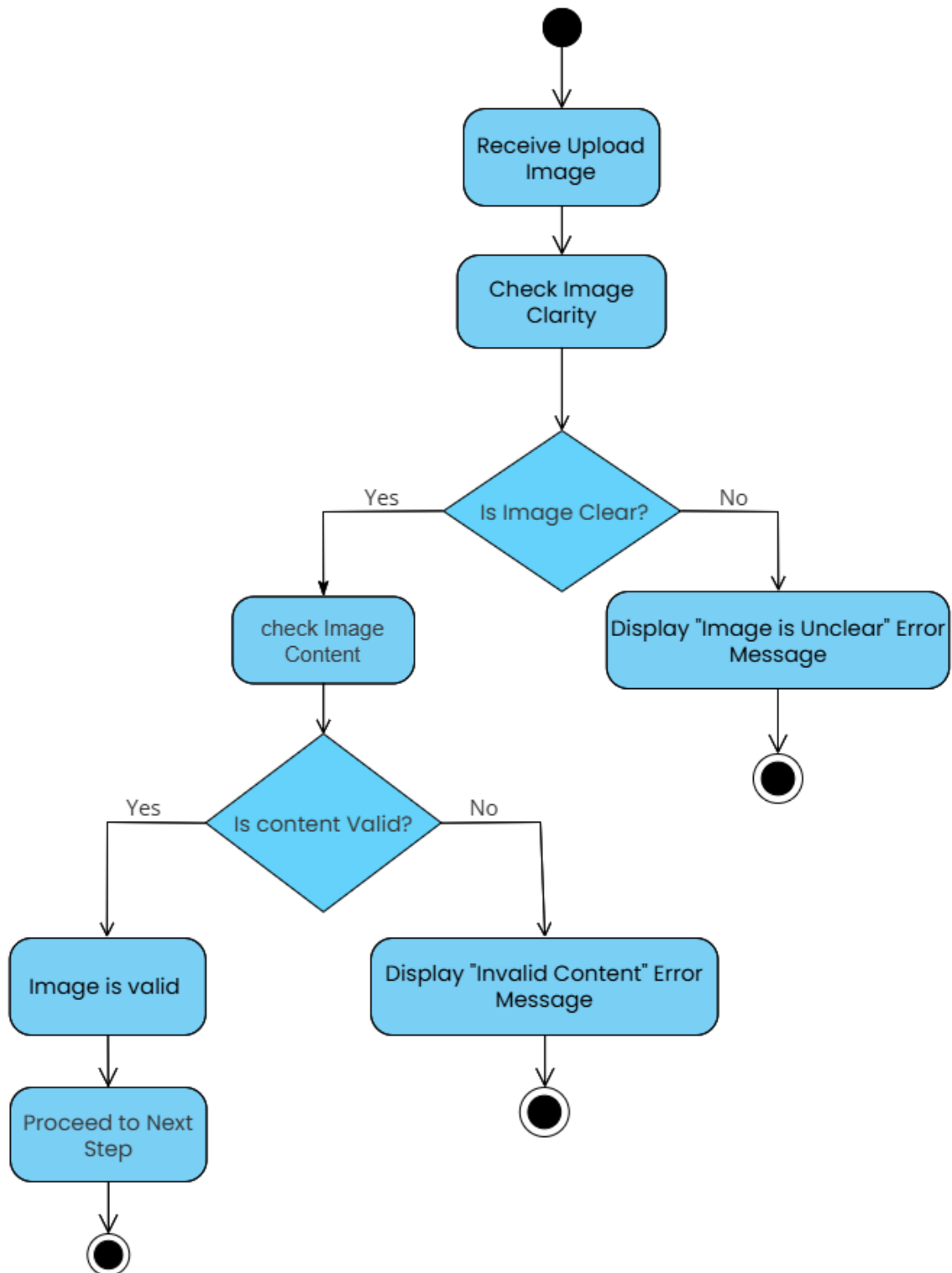


Figure 4.18 Activity Diagram Image Validation

4.5.4 Resize and Normalize

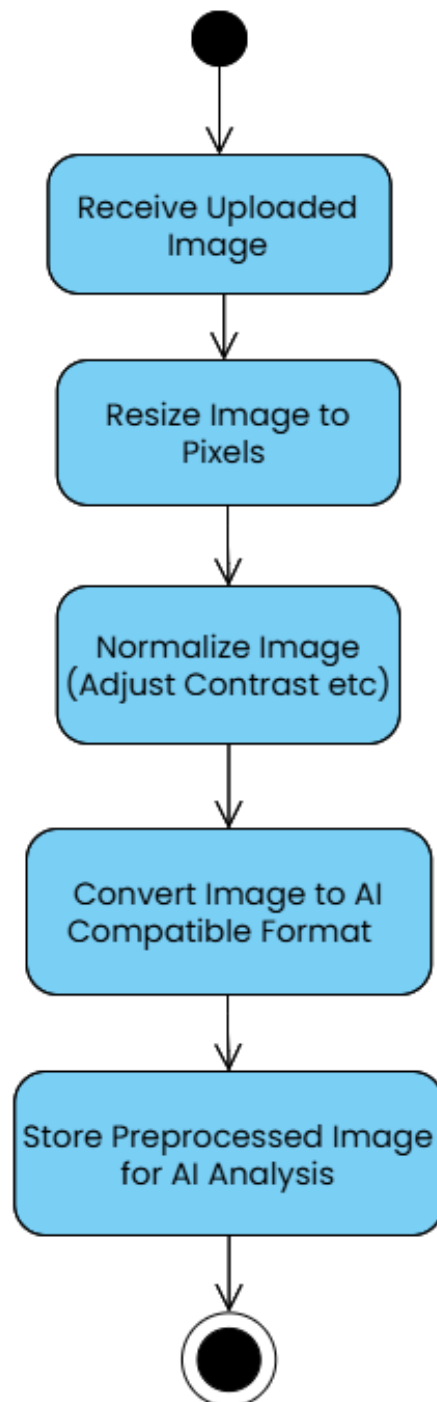


Figure 4.19 Activity Diagram Resize and Normalize

4.5.5 AI Model Analyze

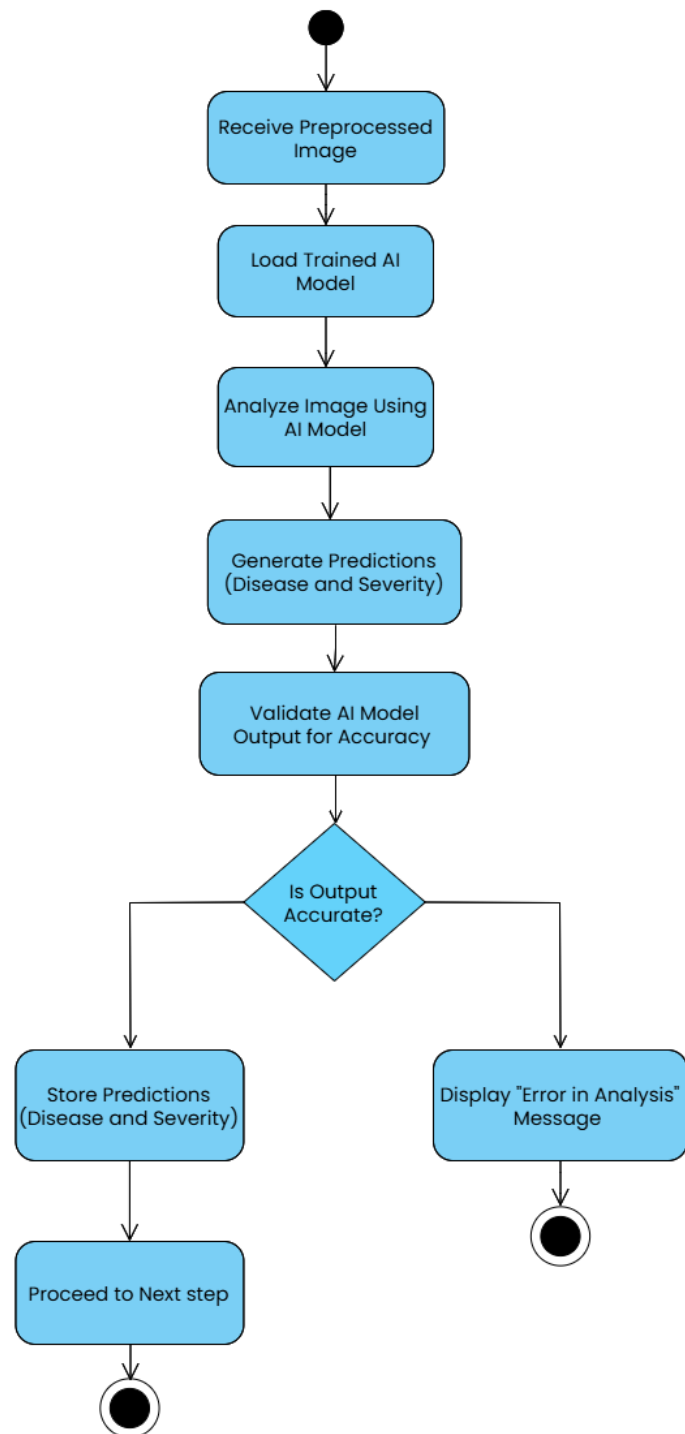


Figure 4.20 Activity Diagram AI Model Analyze

4.5.6 Retrieve Local Weather Data

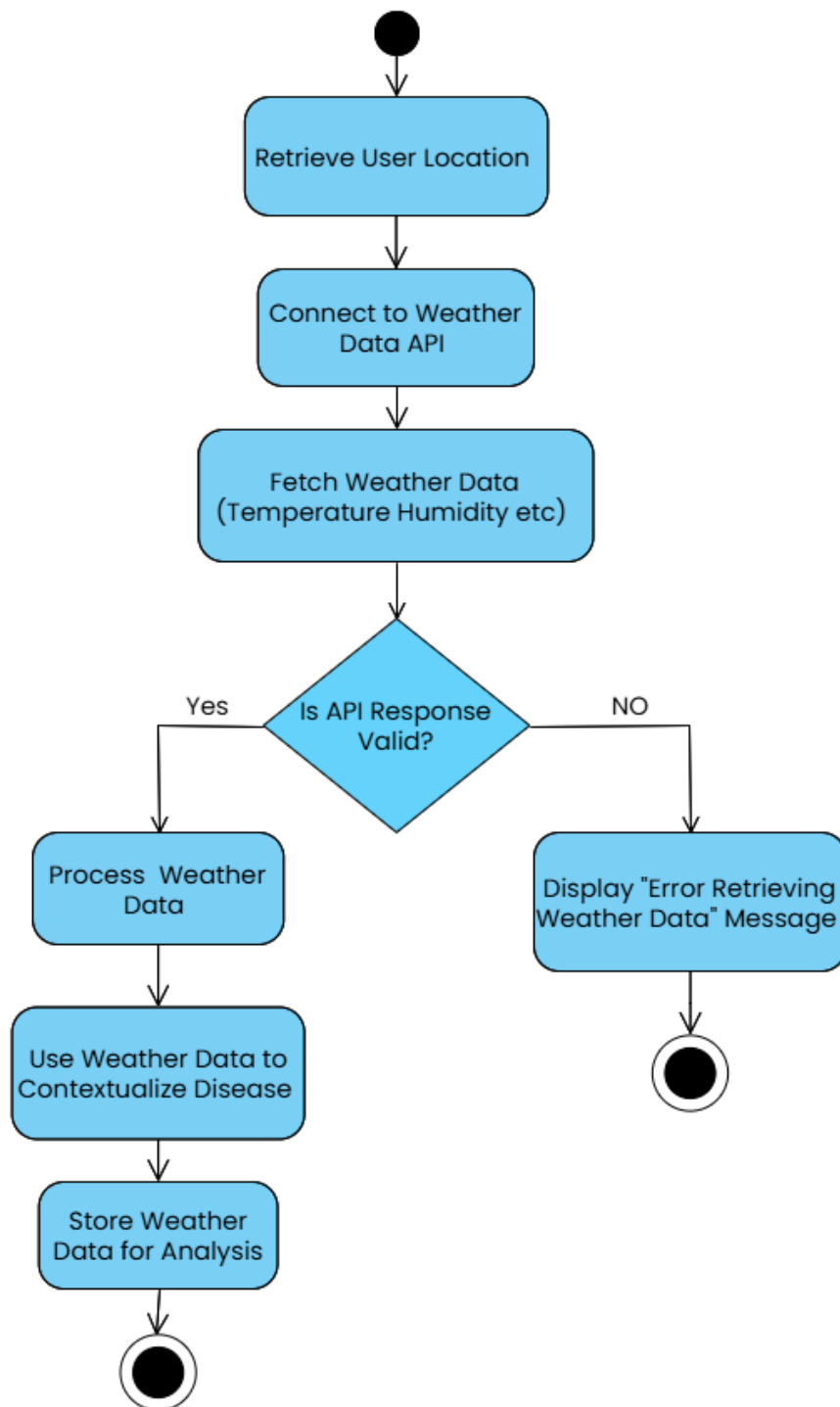


Figure 4.21 Activity Diagram Retrieve Local Weather Data

4.5.7 Disease and Severity Detection

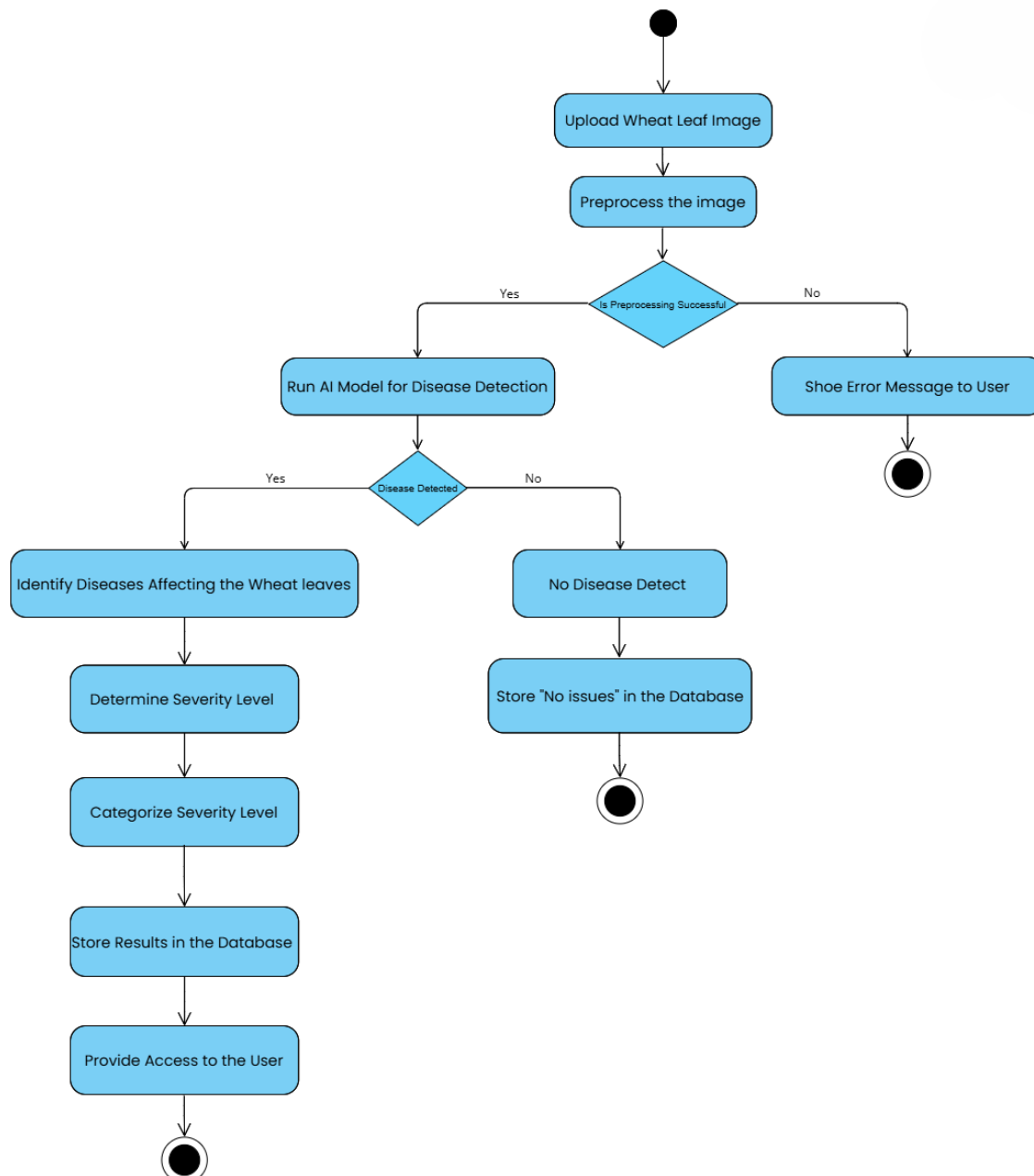


Figure 4.22 Activity Diagram Disease and Severity Detection

4.5.8 Generate Treatment Recommendations

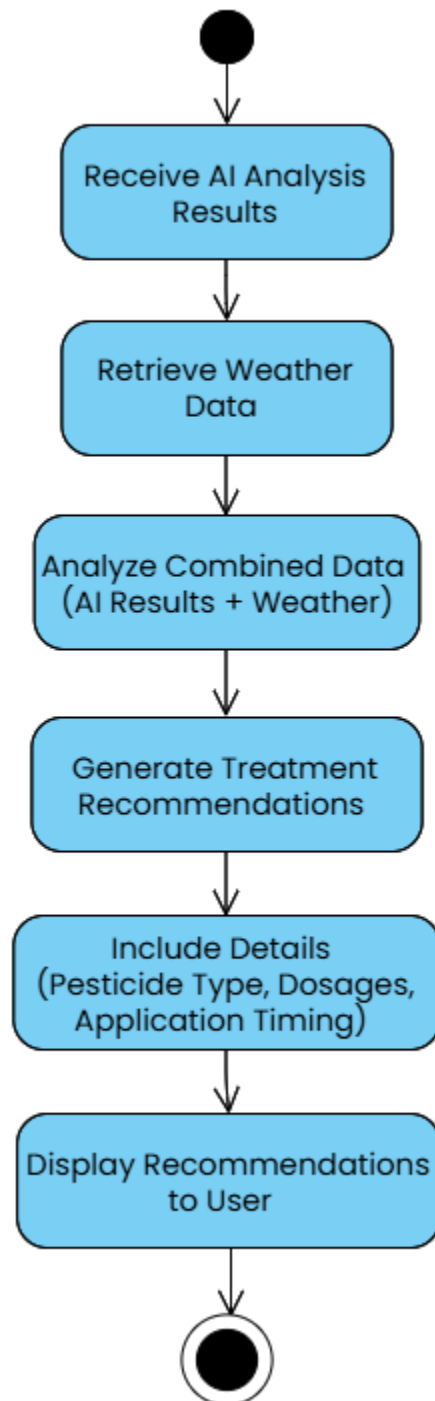


Figure 4.23 Activity Diagram Generate Treatment Recommendations

4.5.9 Display Result

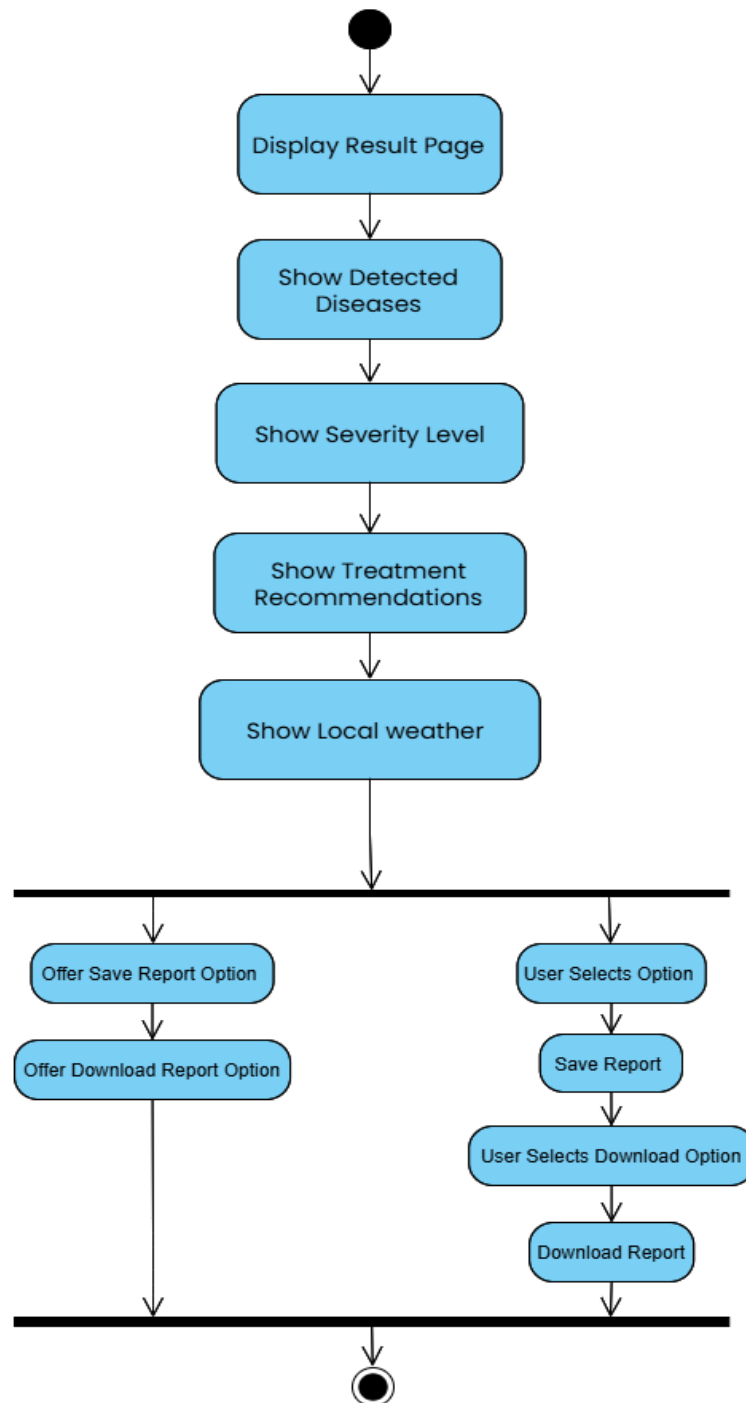


Figure 4.24 Activity Diagram Display Result

4.5.10 Aggregated Activity Diagram

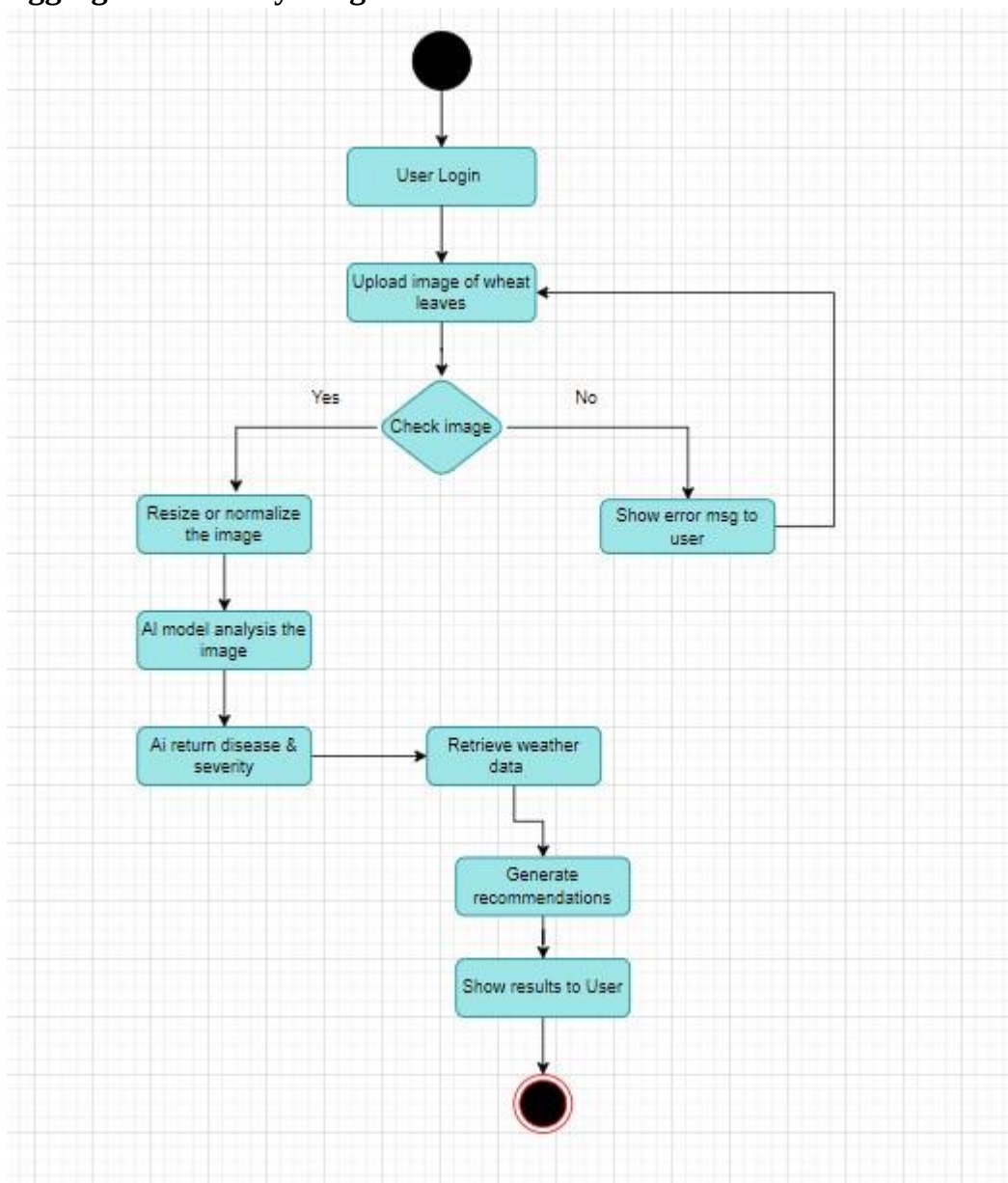


Figure 4.25 Aggregated Activity Diagram

4.6 Sequence Diagram

4.6.1 User Interface

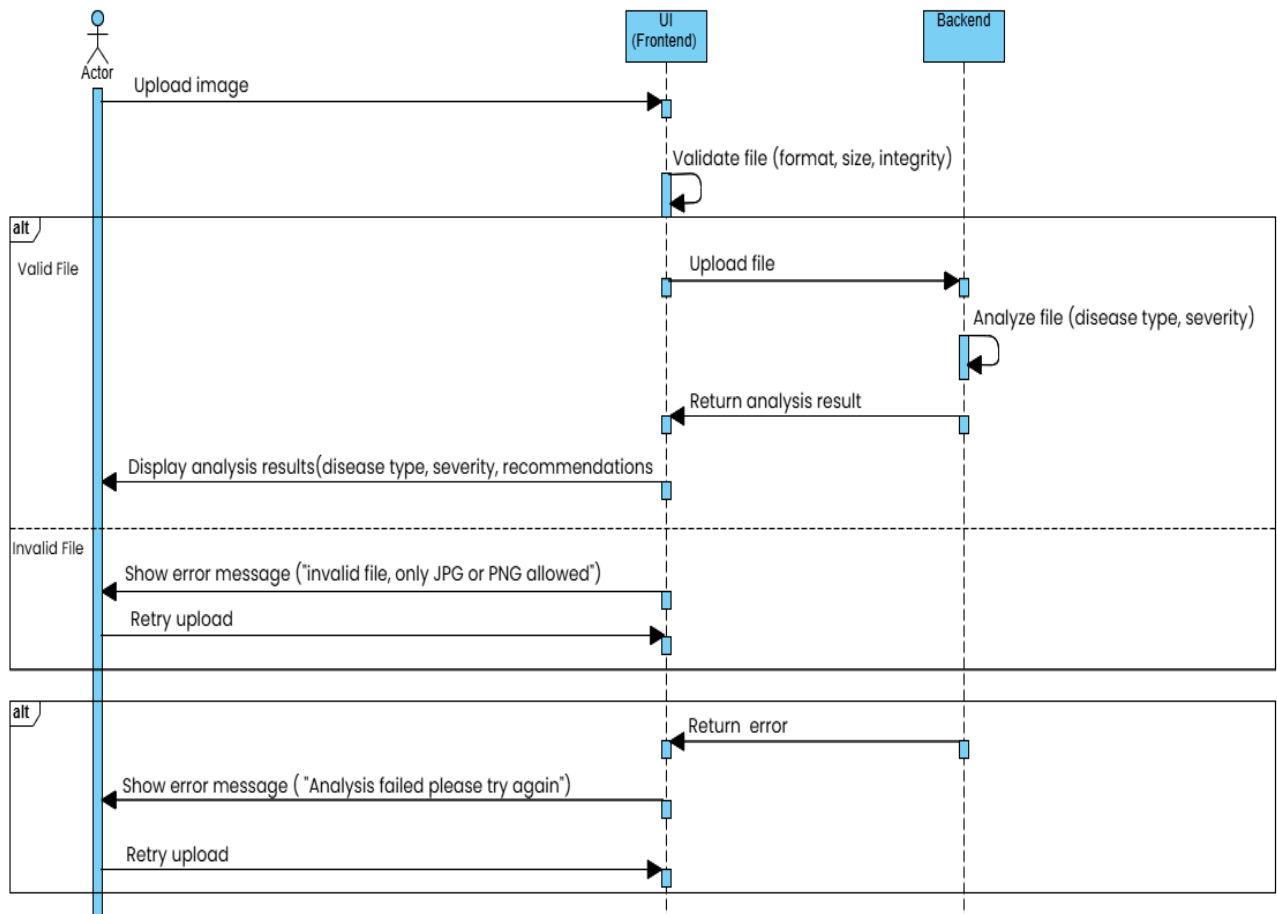


Figure 4.26 Sequence Diagram User Interface

4.6.2 Backend

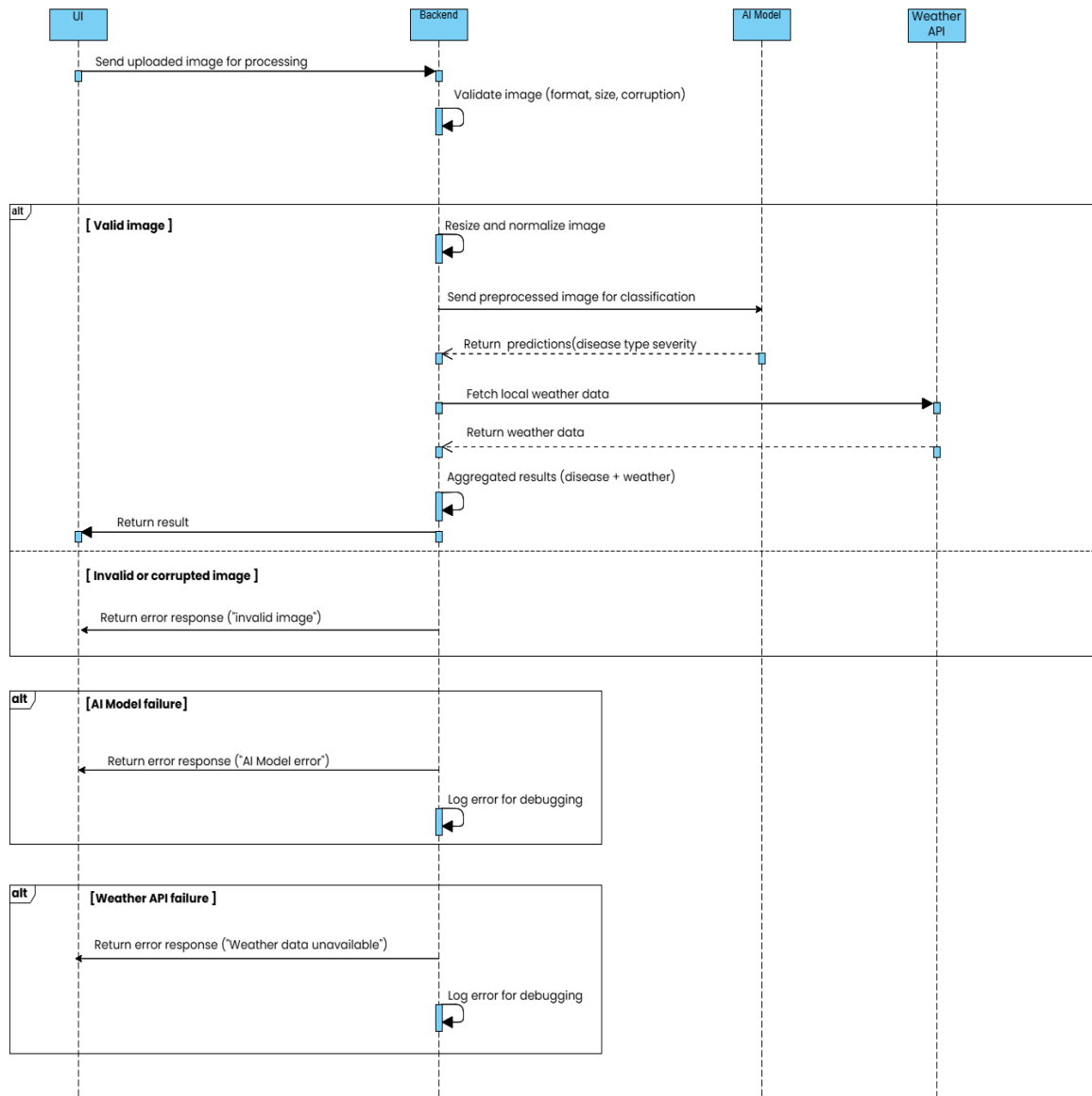


Figure 4.27 Sequence Diagram Backend

4.6.3 AI Model

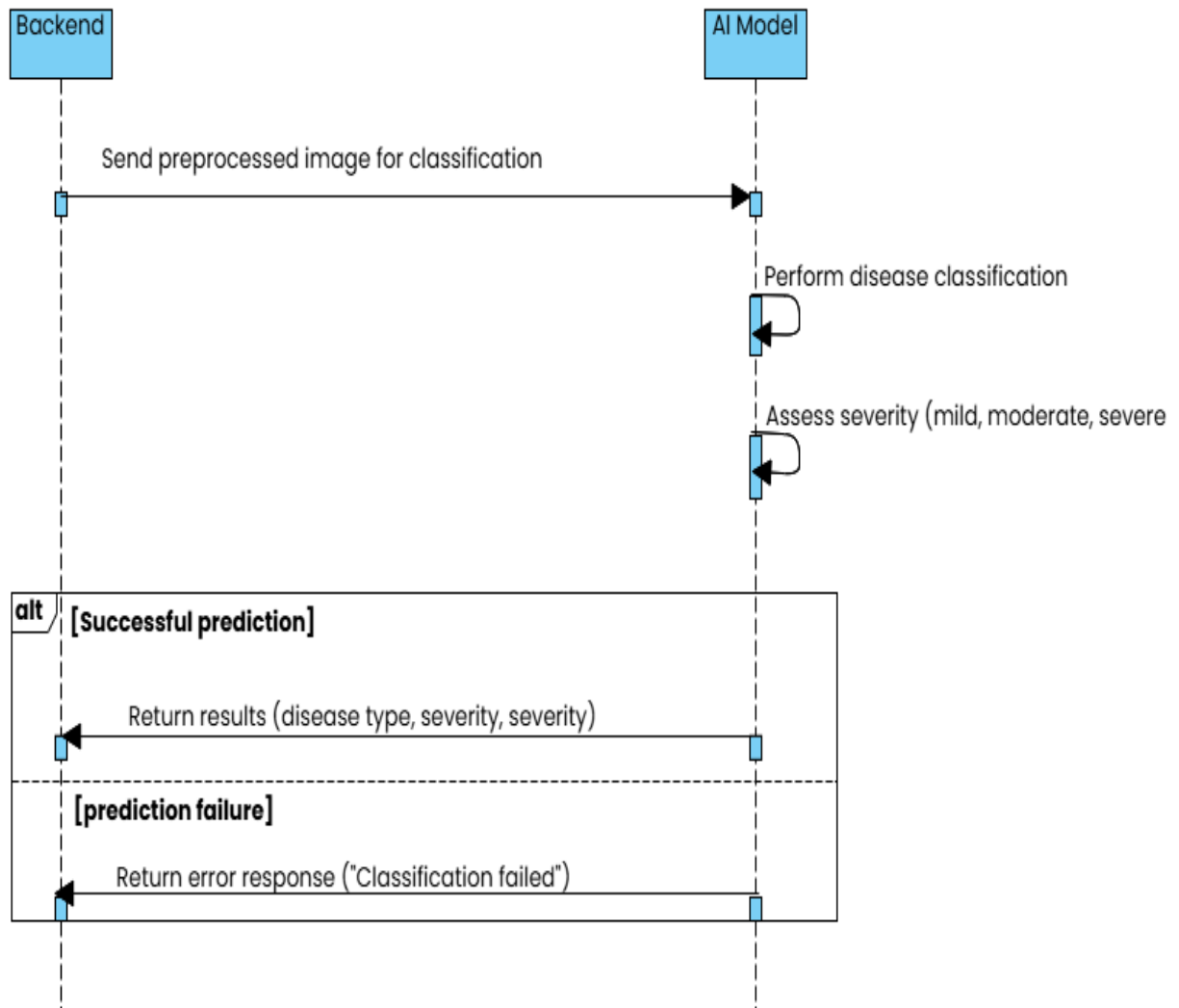


Figure 4.28 Sequence Diagram AI Model

4.6.4 Weather

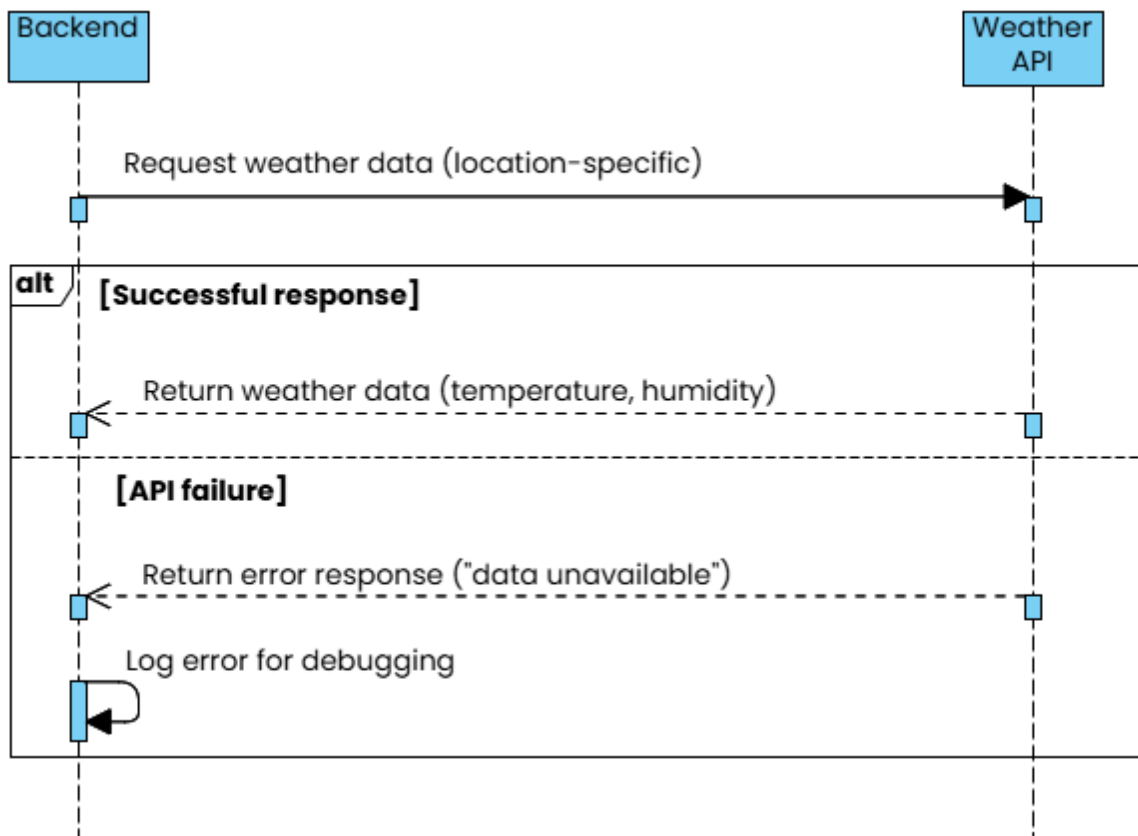


Figure 4.29 Sequence Diagram Weather

4.6.5 Result

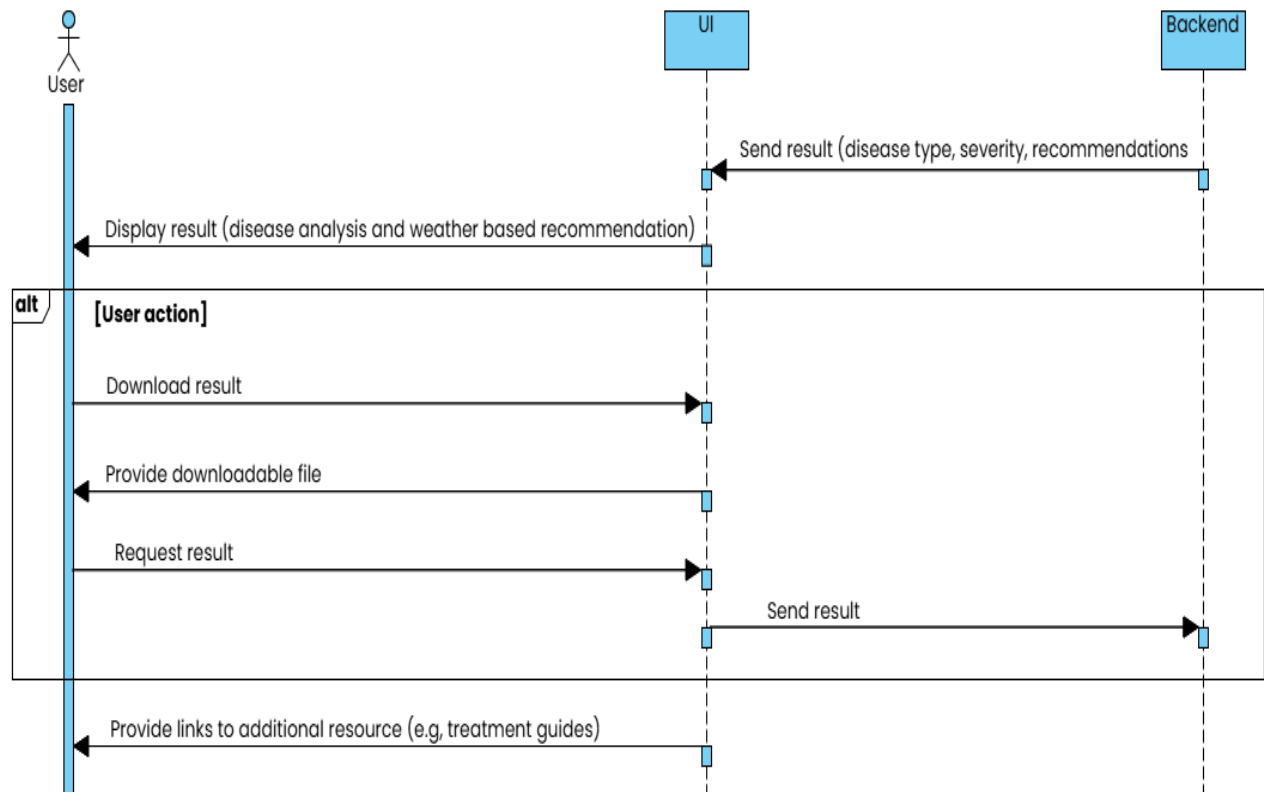


Figure 4.30 Sequence Diagram Result

4.6.6 Aggregated Sequence Diagram

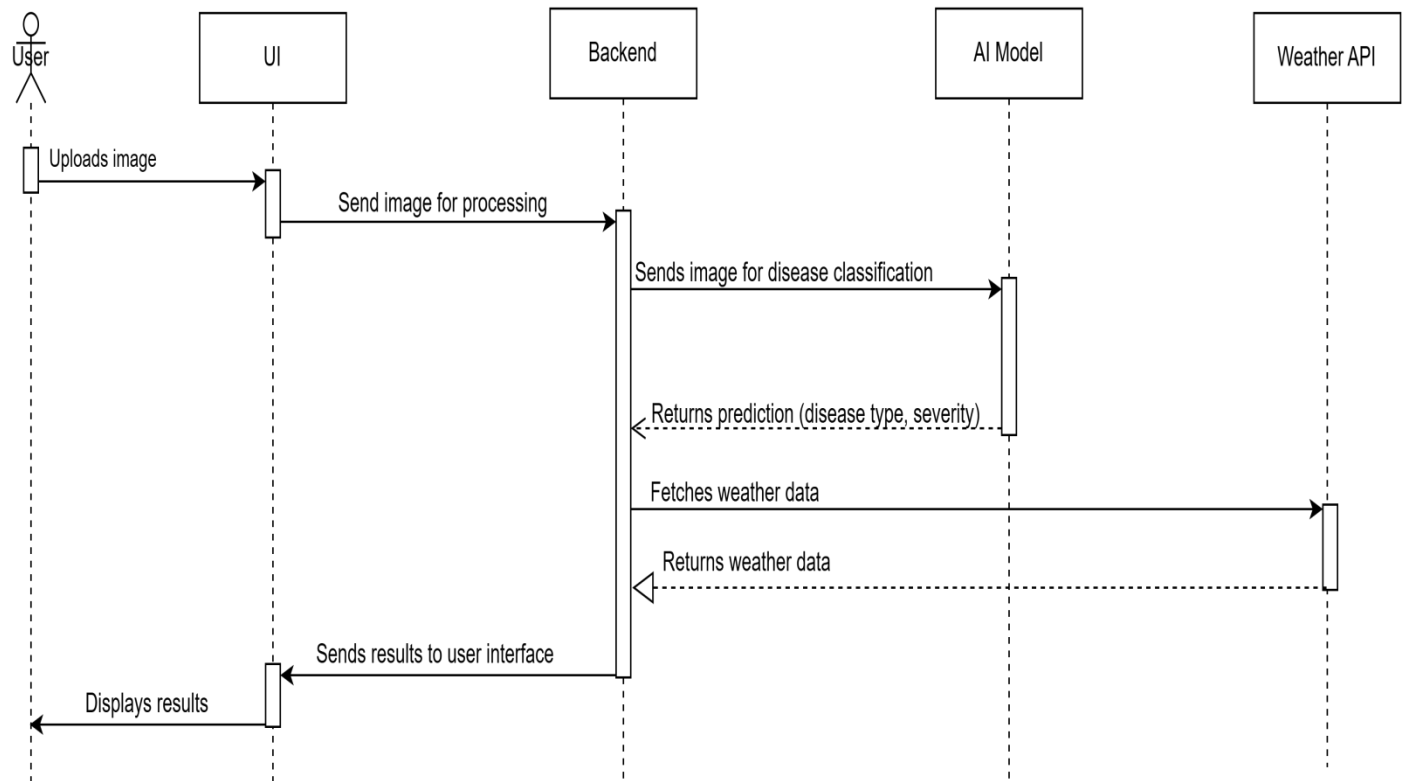


Figure 4.31 Sequence Diagram Aggregated Sequence Diagram

4.7 Component Diagram

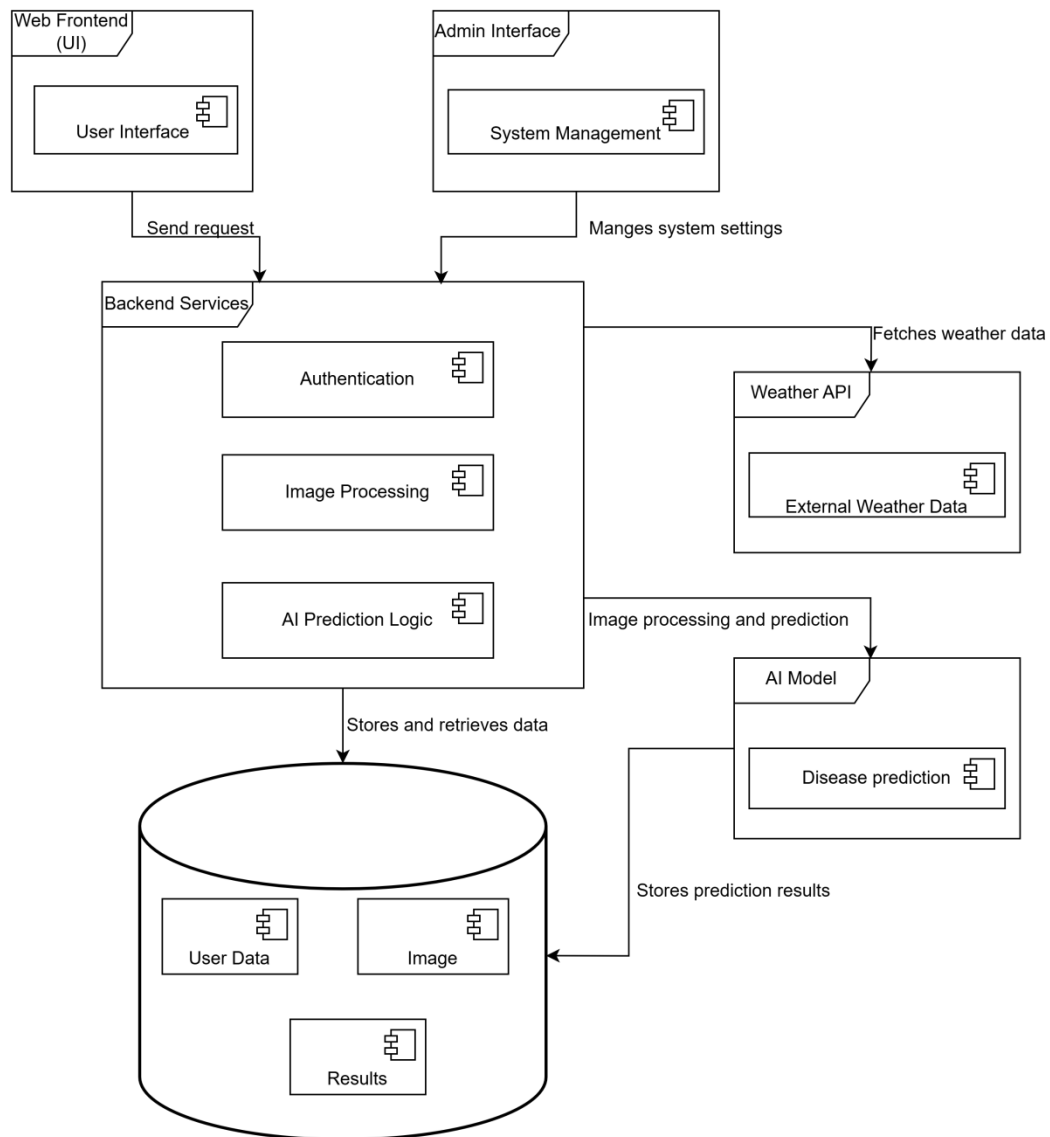


Figure 4.32 Component Diagram

4.8 Deployment Diagram

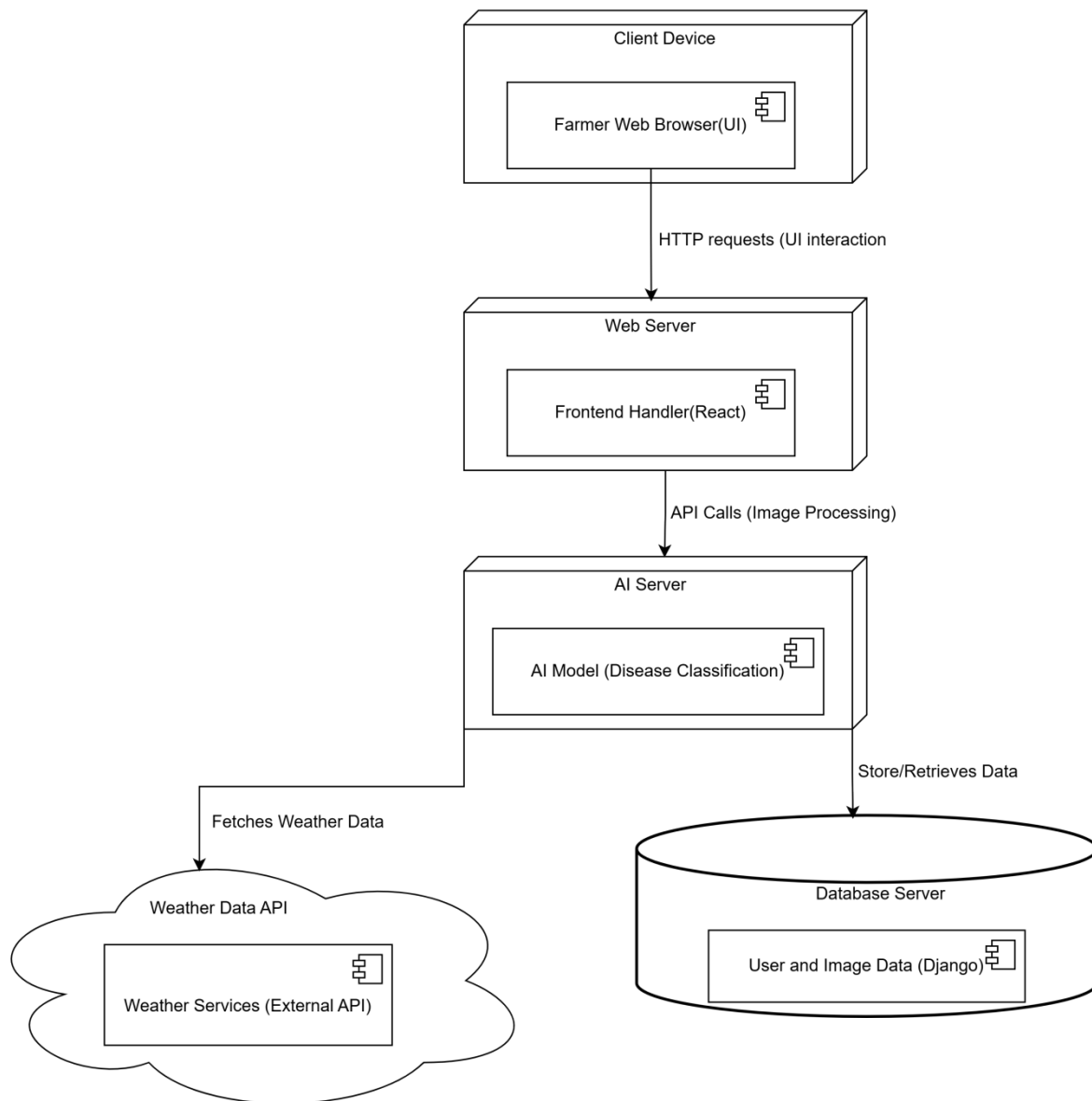


Figure 4.33 Deployment Diagram

4.9 State Transition Diagram

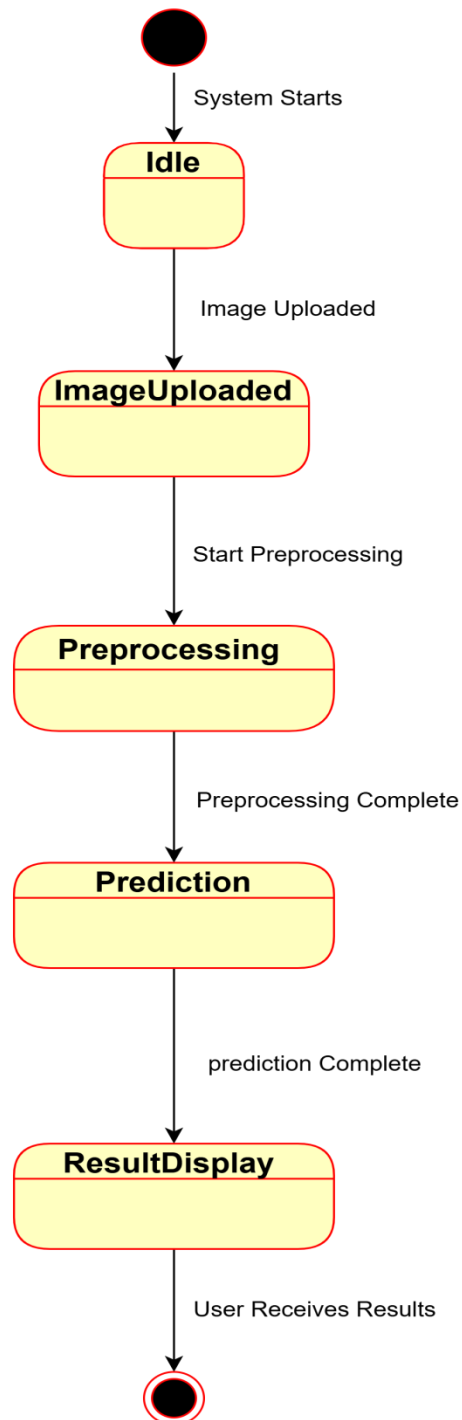


Figure 4.34 State Transition Diagram

4.10 Collaboration Diagram

4.10.1 User Authentication

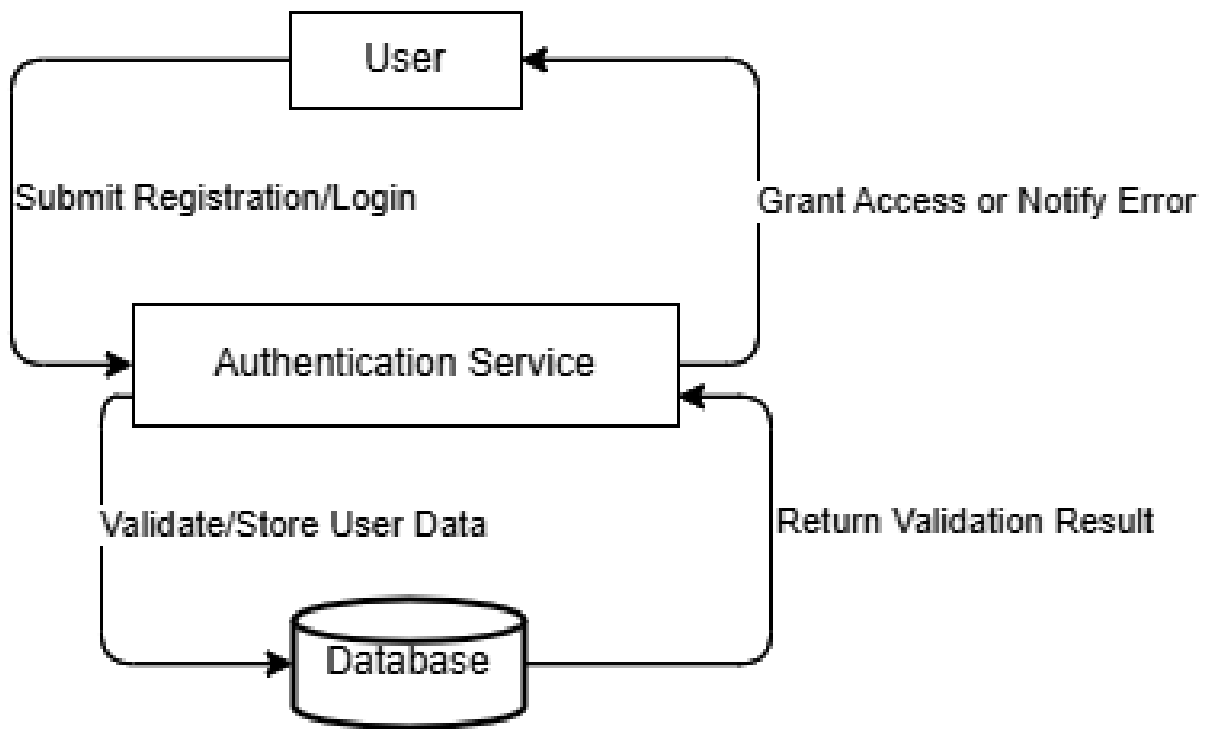


Figure 4.35 Collaboration Diagram User Authentication

4.10.2 Image Upload and Processing

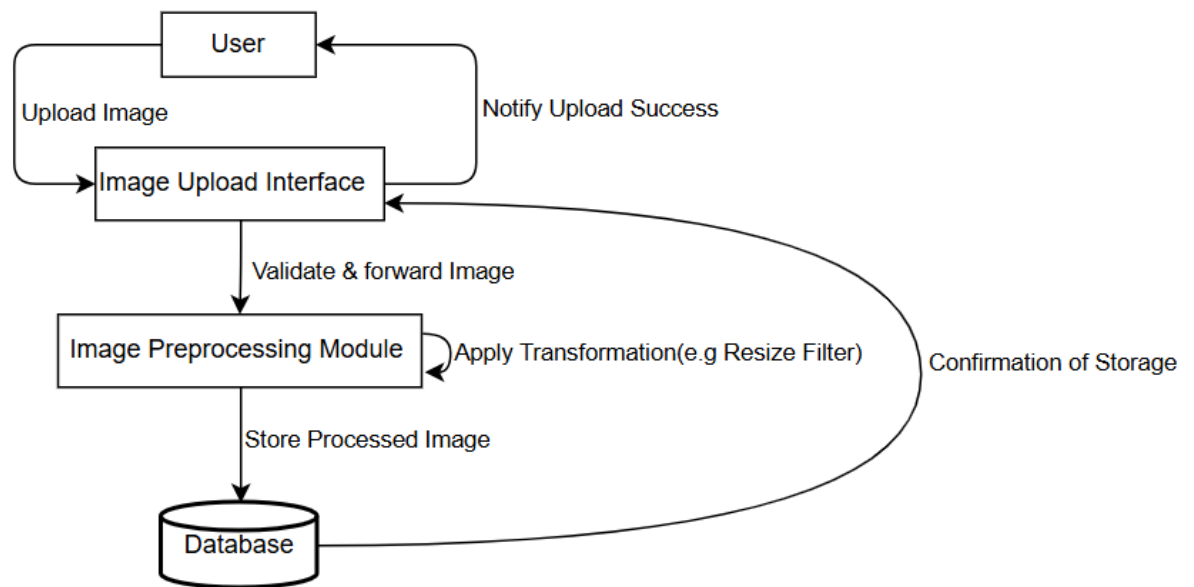


Figure 4.36 Collaboration Diagram Image Upload and Processing

4.10.3 Disease Identification

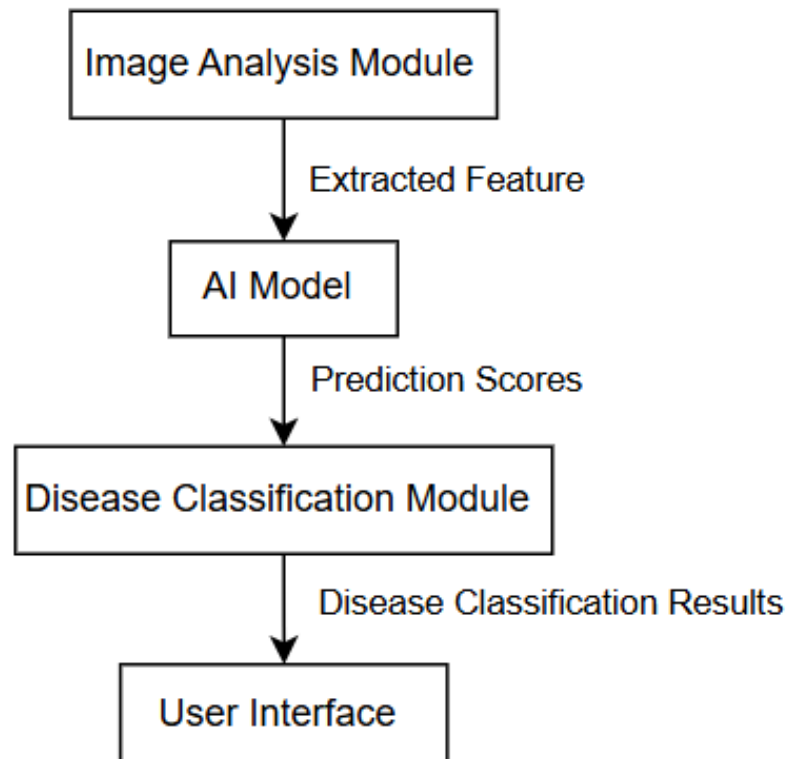


Figure 4.37 Collaboration Diagram Disease Identification

4.10.4 Disease Management

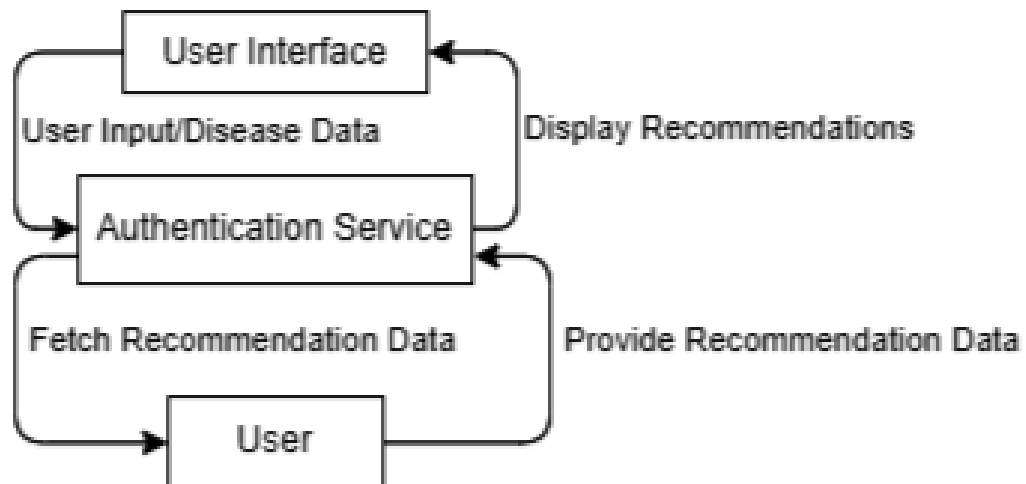


Figure 4.38 Collaboration Diagram Disease Management

4.10.5 Predictive Analytic

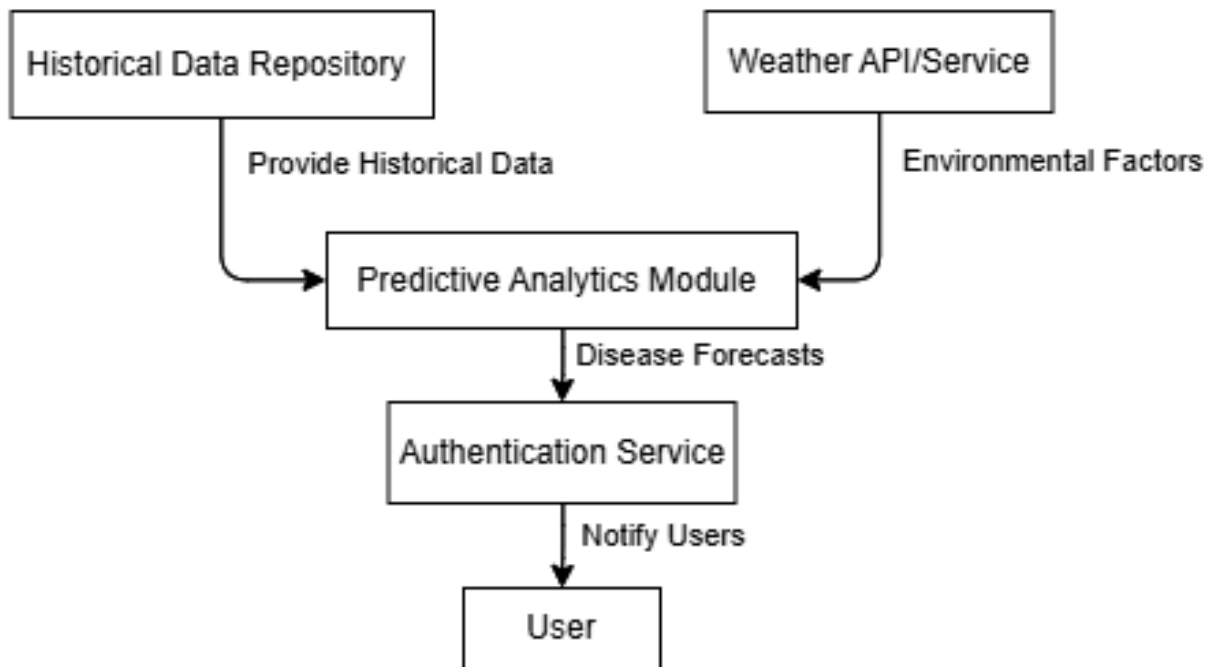


Figure 4.39 Collaboration Diagram Predictive Analytics

4.10.6 Alert and Notification

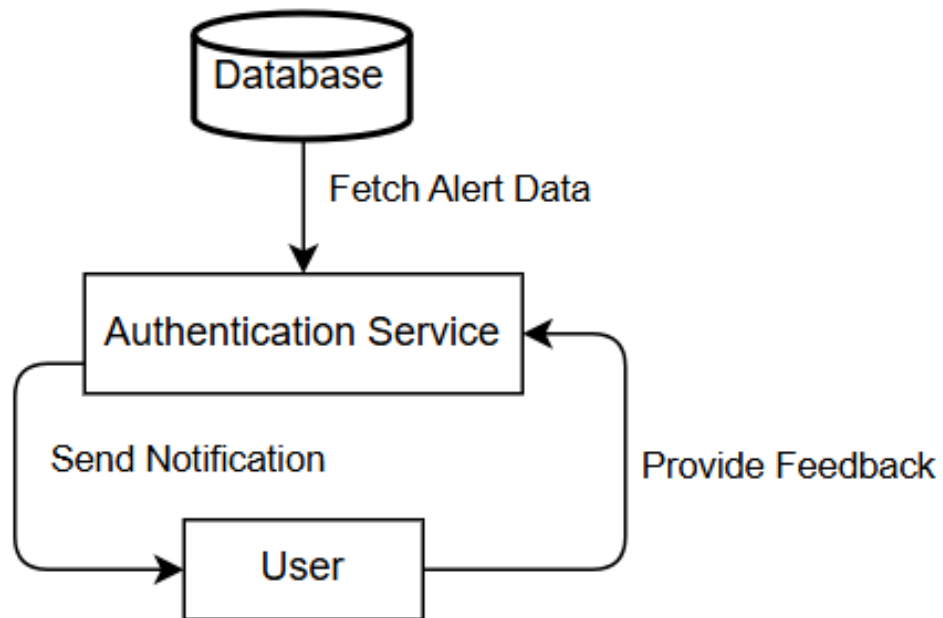


Figure 4.40 Collaboration Diagram Alert and Notification

4.10.7 Aggregated Collaboration Diagram

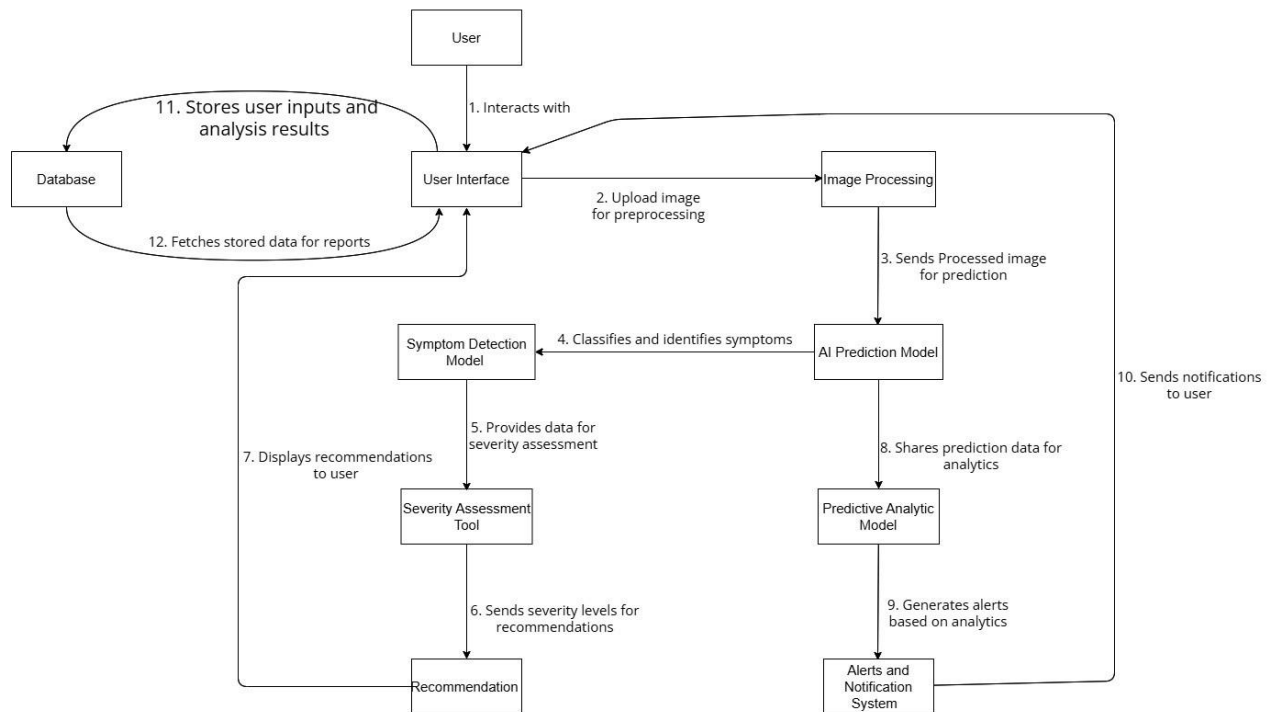


Figure 4.41 Aggregated Collaboration Diagram

CHAPTER 5

Testing

5.1 Test Case Specifications

This Testing phase includes all the Test Cases of the functional requirements of your project.

- 1.User Register
- 2.User Login
- 3.User Forget Password
- 4.User Logout
- 5.User Connects to Database
- 6.User Upload image
- 7.Image processing
- 8.Visualize Results
- 9.Recommendation

5.1.1 User Register

Table 5.24: Test Case Specification signup successful

Positive Test Case	
ID	TC_USER_REGISTRATION_SUCCESS
Priority	High
Description	Verify that a user can register with valid details.
Reference	FR-01 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	New User
Pre-requisites	System online, internet connection available
Steps	Launch the application Click "Sign Up" button Enter valid details (name, email, password) Click "Register"
Input	Valid email, password
Expected result	User account is created successfully.
Status	Tested, passed

Table 5.25: Test Case Specification signup failure

Negative Test Case	
ID	TC_USER_REGISTRATION_INVALID_INPUT
Priority	High
Description	Verify system shows error on invalid registration input
Reference	FR-01 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	New User
Pre-requisites	System online, internet connection available
Steps	Launch application Click "Sign Up" Enter invalid email or short password
Input	Invalid email/password
Expected result	Error message displayed; registration fails
Status	Tested, passed

5.1.2 User Login

Table 5.26: Test Case Specification login successful

Positive Test Case	
ID	TC_USER_LOGIN_SUCCESS
Priority	High2
Description	Verify user can log in with correct credentials
Reference	FR-02 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	User registered
Steps	Open application Click "Login" Enter valid credentials Click "Submit"
Input	Valid email and password
Expected result	User logged in successfully
Status	Tested, passed

Table 5.27: Test Case Specification login failure

Negative Test Case	
ID	TC_USER_LOGIN_FAILURE
Priority	High
Description	Login fails with incorrect credentials
Reference	FR-02 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	System online
Steps	Click "Login" Enter incorrect password/email Submit
Input	Wrong email/password
Expected result	Error: Invalid login
Status	Tested, passed

5.1.3 User Forget Password

Table 5.28: Test Case Specification forget password successful

Positive Test Case	
ID	TC_FORGOT_PASSWORD_SUCCESS
Priority	Medium
Description	Valid password recovery process
Reference	FR-03 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Internet access, register Email
Steps	Click "Forgot Password" Enter registered email Submit
Input	Valid registered email
Expected Result	Password reset link is sent
Status	Tested, passed

Table 5.29: Test Case Specification forget password failure

Negative Test Case	
ID	TC_FORGOT_PASSWORD_FAILURE
Priority	Medium
Description	Invalid email entered
Reference	FR-03 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Any user
Pre-requisites	Internet access, register Email
Steps	Click "Forgot Password" Enter unregistered email Submit
Input	Unregistered email
Expected Result	"Email not found" error
Status	Tested, passed

5.1.4 User Logout

Table 5.30: Test Case Specification logout successful

Positive Test Case	
ID	TC_USER_LOGOUT_SUCCESS
Priority	Medium
Description	User logs out successfully
Reference	FR-04 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Logged-in User
Pre-requisites	User logged in
Steps	Click "Logout"
Input	None
Expected Result	User is redirected to login screen
Status	Tested, passed

Table 5.31: Test Case Specification logout failure

Negative Test Case	
ID	TC_USER_LOGOUT_FAILURE
Priority	Low
Description	Attempting to logout without being logged in
Reference	FR-04 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Guest
Pre-requisites	Not logged in
Steps	Open application Click "Logout"
Input	None
Expected Result	System prompts for login or shows error message
Status	Tested, passed

5.1.5 User Connects to Database

Table 5.32: Test Case Specification Connects to Database successful

Positive Test Case	
ID	TC_DATABASE_CONNECT_SUCCESS
Priority	High
Description	Successful DB connection
Reference	FR-05 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	System
Pre-requisites	DB is online
Steps	Launch application Connect to DB
Input	Connection credentials
Expected Result	DB connection established
Status	Tested, passed

Table 5.33: Test Case Specification Connects to Database failure

Negative Test Case	
ID	TC_DATABASE_CONNECT_FAILURE
Priority	High
Description	DB connection fails
Reference	FR-05 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	System
Pre-requisites	No internet or DB down
Steps	Attempt DB connection
Input	Invalid credentials
Expected Result	Error: DB connection failed
Status	Tested, passed

5.1.6 User Upload image

Table 5.34: Test Case Specification Upload image successful

Positive Test Case	
ID	TC_IMAGE_UPLOAD_SUCCESS
Priority	High
Description	User uploads a valid image
Reference	FR-06 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Logged in, image available
Steps	Click "Upload" Choose image Submit
Input	Valid image file
Expected Result	Image is uploaded
Status	Tested, passed

Table 5.35: Test Case Specification Upload image failure

Negative Test Case	
ID	TC_IMAGE_UPLOAD_FAILURE
Priority	High
Description	Uploading invalid image file
Reference	FR-06 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Logged in
Steps	Attempt to upload unsupported format
Input	Non-image file
Expected Result	Error message shown
Status	Tested, passed

5.1.7 Image processing

Table 5.36: Test Case Specification Image processing successful

Positive Test Case	
ID	TC_IMAGE_PROCESS_SUCCESS
Priority	High
Description	Image is processed for disease detection
Reference	FR-07 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Valid image uploaded
Steps	Click "Analyze"
Input	Uploaded image
Expected Result	Disease detection result generated
Status	Tested, passed

Table 5.37: Test Case Specification Image processing failure

Negative Test Case	
ID	TC_IMAGE_PROCESS_FAILURE
Priority	High
Description	Image processing fails due to corrupted or incompatible image
Reference	FR-07 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Invalid image uploaded
Steps	Click "Analyze"
Input	Corrupted or unsupported image file
Expected Result	Error: Unable to process image
Status	Tested, passed

5.1.8 Visualize Results

Table 5.38: Test Case Specification Visualize Results successful

Positive Test Case	
ID	TC_VISUALIZE_RESULT_SUCCESS
Priority	Medium
Description	Visual display of analysis result
Reference	FR-08 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Image processed
Steps	Click "View Results"
Input	Processed data
Expected Result	Graph or highlighted area shown
Status	Tested, passed

Table 5.39: Test Case Specification Visualize Results failure

Negative Test Case	
ID	TC_VISUALIZE_RESULT_FAILURE
Priority	Medium
Description	Visualization fails due to missing or malformed data
Reference	FR-08 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	No processed image available
Steps	Click "View Results"
Input	Missing or invalid data
Expected Result	Error: Cannot visualize result
Status	Tested, passed

5.1.9 Recommendation

Table 5.40: Test Case Specification Recommendation successful

Positive Test Case	
ID	TC_RECOMMENDATION_SUCCESS
Priority	Medium
Description	System gives proper disease treatment advice
Reference	FR-09 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	Image processed
Steps	Click "Get Recommendation"
Input	Disease result
Expected Result	Suggestion for treatment and prevention
Status	Tested, passed

Table 5.41: Test Case Specification Recommendation failure

Negative Test Case	
ID	TC_RECOMMENDATION_FAILURE
Priority	Medium
Description	System fails to generate recommendation due to missing analysis data
Reference	FR-09 (SmartRust AI for Wheat Disease Detection and Prevention)
Users	Registered User
Pre-requisites	No processed image available
Steps	Click "Get Recommendation"
Input	None or invalid result
Expected Result	Error: Unable to generate recommendation
Status	Tested, passed

5.2 Black Box Test Cases

5.2.1 Equivalence Partitions

5.2.1.1 User register

Table 5.42: Equivalence Partitions User register

Input Field	Valid Classes	Invalid Classes
Email	Proper format (e.g., user@example.com), not already registered	Malformed email, already used email, empty field
Password	6–50 characters, includes letters, digits, symbols	Less than 6 or more than 50 characters, empty field
Name	At least 1 character	Empty field

5.2.1.2 login

Table 5.43: Equivalence Partitions Login

Input Field	Valid Classes	Invalid Classes
Email	Registered and well-formed email	Unregistered email, malformed email, empty field
Password	Correct password	Incorrect password, empty field

5.2.1.3 Forget password

Table 5.44: Equivalence Partitions Forget password

Input Field	Valid Classes	Invalid Classes
Email	Registered email	Unregistered or empty email

5.2.1.4 User logout

Table 5.45: Equivalence Partitions User logout

Input Field	Valid Classes	Invalid Classes
Session Token	Active user session	No active session, expired session

5.2.1.5 Connect to database

Table 5.46: Equivalence Partitions Connect to database

Input Field	Valid Classes	Invalid Classes
DB URL	Well-formed URL with valid credentials	Invalid URL, missing credentials, offline state
Username/Password	Correct credentials	Incorrect credentials, empty fields

5.2.1.6 Upload image

Table 5.47: Equivalence Partitions Upload image

Input Field	Valid Classes	Invalid Classes
Image File	JPEG/PNG, <10MB, uncorrupted	.txt/.pdf, >10MB, corrupted file, unsupported format, empty

5.2.1.7 Image process

Table 5.48: Equivalence Partitions Image process

Input Field	Valid Classes	Invalid Classes
Uploaded Image	Valid, supported image uploaded	No image uploaded, corrupted or unsupported file

5.2.1.8 Visualize results

Table 5.49: Equivalence Partitions Visualize results

Input Field	Valid Classes	Invalid Classes
Detection Output	Analysis completed and results available	Results not generated no image processed

5.2.1.9 Recommendation

Table 5.50: Equivalence Partitions Recommendation

Input Field	Valid Classes	Invalid Classes
Disease Detection	Detection complete, disease identified	No detection data, processing skipped

5.2.2 Boundary Value Analysis

5.2.2.1 User Registration

Table 5.51: Boundary Value Analysis User Registration

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
User Registration	Email Length	0 characters	Invalid	Show "email required" error
		1–254 characters	Valid	Accepted
		255 characters	Invalid	Show "email too long" error
	Password Length	5 characters	Invalid	Show "password too short" error
		6–50 characters	Valid	Account created successfully
		51 characters	Invalid	Show "password too long" error
	Name Length	0 characters	Invalid	Show "name required" error
		1–100 characters	Valid	Account created successfully
		101 characters	Invalid	Show "name too long" error

5.2.2.2 User Login

Table 5.52: Boundary Value Analysis User Login

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
User Login	Email Length	0 characters	Invalid	Show "email required" error
		1–254 characters	Valid	Login proceeds if password is also valid
		255 characters	Invalid	Show "email too long" error
	Password Length	0–5 characters	Invalid	Show "password too short" error
		6–50 characters	Valid	Login proceeds if email is valid
		51 characters	Invalid	Show "password too long" error

5.2.2.3 Forget Password

Table 5.53: Boundary Value Analysis Forget Password

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
Forget Password	Email Length	0 characters	Invalid	Show "email required" error
		1–254 characters	Valid	Send reset link if email is registered
		255 characters	Invalid	Show "email too long" error

5.2.2.4 User Logout

Table 5.54: Boundary Value Analysis User Logout

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
User Logout	Access After Logout	Try direct URL access	Invalid	Redirect to login (unauthorized access)
		Click logout from dashboard	Valid	Session cleared, redirected to login

5.2.2.5 Connect Database

Table 5.55: Boundary Value Analysis Connect Database

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
Connect Database	Port Number	-1	Invalid	Show "invalid port" error
	File Name Length	0–65535	Valid	Connection successful
		65536	Invalid	Show "invalid port" error

5.2.2.6 Image Upload

Table 5.56: Boundary Value Analysis Image Upload

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
Image Upload	Image Size	-0.1 MB	Invalid	Show "invalid file size" error
		0–10 MB	Valid	Image accepted
		10.1 MB	Invalid	Show "file too large" error
	Filename Length	0 characters	Invalid	Show "filename required" error
		1–255 characters	Valid	File accepted
		256 characters	Invalid	Show "filename too long" error

5.2.2.7 Image Processing

Table 5.57: Boundary Value Analysis Image Processing

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
Image Processing	Disease Severity %	-1%	Invalid	Show "invalid severity" error
		0–100%	Valid	Show results
		101%	Invalid	Show "value out of range" error
	Confidence %	-1%	Invalid	Show "invalid confidence" error
		0–99%	Valid	Show prediction results
		100%	Invalid or Edge	Requires manual verification

5.2.2.8 Visualize Results

Table 5.58: Boundary Value Analysis Visualize Results

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
Visualize Results	Image Processed Flag	False	Invalid	Show "no result available" error
		True	Valid	Show visual indicators and results

5.2.2.9 Recommendation

Table 5.59: Boundary Value Analysis Recommendation

Module	Input Field	Boundary Values Tested	Valid/Invalid	Expected Result
Recommendation	Processed Result Exists	False	Invalid	Show "no data to recommend" error
		True	Valid	Show treatment recommendation

5.2.3 Decision Table Testing

5.2.3.1 User Register

Table 5.60: Decision Table Testing User Register

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check for Active Internet Connection	T	F	F	F
C2	Check for Valid Email Format	T	F	F	T
C3	Check for Valid Password	T	F	T	F
C4	Check for All Required Fields	T	F	T	T
A1	Registration Successful	Execute			
A2	Message “No Internet Connection”		Execute		
A3	Message “Invalid Email”			Execute	
A4	Message “Invalid Password”				Execute

5.2.3.2 Login

Table 5.61: Decision Table Testing Login

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check for Registered Email	T	F	T	T
C2	Check for Correct Password	T	F	F	T
C3	Check for Account Status	T	F	T	F
A1	Login Successful	Execute			
A2	Message “Email Not Found”		Execute		
A3	Message “Incorrect Password”			Execute	
A4	Message “Account Blocked”				Execute

5.2.3.3 Forget Password

Table 5.62: Decision Table Testing Forget Password

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check for Registered Email	T	F	T	T
C2	Check for Valid Email Format	T	F	F	T
C3	Check for Internet Connectivity	T	F	F	F
A1	Password Reset Email Sent	Execute			
A2	Message “Email Not Registered”		Execute		
A3	Message “Invalid Email Format”			Execute	
A4	Message “No Internet Connection”				Execute

5.2.3.4 Logout

Table 5.63: Decision Table Testing Logout

ID	Condition/Action	Test Case 1	Test Case 2
C1	Check if User is Logged In	T	F
A1	Logout Successful	Execute	
A2	Message “No User Logged In”		Execute

5.2.3.5 Connect to Database

Table 5.64: Decision Table Testing Connect to Database

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check for Active Internet Connection	T	F	T	T
C1	Check for Correct Database Credentials	T	F	F	T
C1	Check if Database Server is Available	T	F	T	F
A1	Database Connection Established	Execute			
A2	Message “No Internet Connection”		Execute		
A3	Message “Invalid Credentials”			Execute	
A4	Message “Database Server Down”				Execute

5.2.3.6 Upload Image

Table 5.65: Decision Table Testing Upload Image

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check for Active Internet Connection	T	F	F	F
C2	Check for Supported File Format (.jpg/.png)	T	F	F	T
C3	Check for File Size Within Limit	T	F	T	F
C4	Check for Non-Empty Image Field	T	F	T	T
A1	Image Uploaded Successfully	Execute			
A2	Message “No Internet Connection”		Execute		
A3	Message “Unsupported File Format”			Execute	
A4	Message “Image Too Large”				Execute

5.2.3.7 Image Processing

Table 5.66: Decision Table Testing Image Processing

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check for Valid Image Data	T	F	T	T
C2	Check for Rust Detection Model Availability	T	T	F	T
C3	Check for Sufficient Image Quality	T	T	T	F
A1	Disease Identification Successful	Execute			
A2	Message “Invalid Image Data”		Execute		
A3	Message “Model Not Available”			Execute	
A4	Message “Image Quality Too Low”				Execute

5.2.3.8 Visualize Results

Table 5.67: Decision Table Testing Visualize Results

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check if Processed Data Available	T	F	T	T
C2	Check if Graph/Chart Libraries Loaded	T	T	F	T
C3	Check if Display Component Working	T	T	T	F
A1	Visualization Displayed Successfully	Execute			
A2	Message “No Processed Data Available”		Execute		
A3	Message “Graph Rendering Failed”			Execute	
A4	Message “Display Component Error”				Execute

5.2.3.9 Recommendation

Table 5.68: Decision Table Testing Recommendation

ID	Condition/Action	Test Case 1	Test Case 2	Test Case 3	Test Case 4
C1	Check if Disease Type Identified	T	F	T	T
C2	Check if Severity Level Assessed	T	T	F	T
C3	Check if Recommendation Rules Exist	T	T	T	F
A1	Display Treatment Recommendation	Execute			
A2	Message “Disease Type Not Identified”		Execute		
A3	Message “Severity Level Unknown”			Execute	
A4	Message “No Recommendation Found”				Execute

5.2.4 State transition Testing

5.2.4.1 User Register

Table 5.69: State transition Testing User Register

State	Event	Transition
Initial	User navigates to registration page	Registration Page Displayed
Registration Page Displayed	User enters valid details and submits	Registration Successful
Registration Page Displayed	User enters invalid details	Registration Page Displayed (Error)
Registration Page Displayed	User cancels registration	Initial
Registration Successful	User receives confirmation email	Initial
Registration Page Displayed (Error)	User corrects errors and resubmits	Registration Successful

5.2.4.2 User Login

Table 5.70: State transition Testing User Login

State	Event	Transition
Initial	User navigates to login page	Login Page Displayed
Login Page Displayed	User enters valid credentials	Login Successful
Login Page Displayed	User enters invalid credentials	Login Page Displayed (Error)
Login Page Displayed	User cancels login	Initial
Login Page Displayed (Error)	User corrects credentials	Login Successful
Login Page Displayed (Error)	User cancels login	Initial

5.2.4.3 User Forget Password

Table 5.71: State transition Testing User Forget Password

State	Event	Transition
Initial	User clicks "Forgot Password"	Forgot Password Page Displayed
Forgot Password Page Displayed	User submits valid email	Token Sent
Forgot Password Page Displayed	User submits invalid email	Forgot Password Page (Error)
Forgot Password Page Displayed	User cancels reset	Initial
Forgot Password Page (Error)	User corrects email and resubmits	Token Sent
Token Sent	User sets new password via token	Password Reset Successful
Password Reset Successful	User logs in again	Login Page Displayed

5.2.4.4 User Logout

Table 5.72: State transition Testing User Logout

State	Event	Transition
Logged In	User clicks logout	Logged Out
Logged Out	User navigates to login	Login Page Displayed

5.2.4.5 User Connects to Database

Table 5.73: State transition Testing Connects to Database

State	Event	Transition
Application Started	User attempts DB connection	DB Connected
Application Started	Connection fails	DB Connection Error
DB Connection Error	User retries connection	DB Connected

5.2.4.6 User Upload image

Table 5.74: State transition Testing Upload image

State	Event	Transition
DB Connected	User navigates to upload page	Upload Page Displayed
Upload Page Displayed	User uploads valid image	Image Uploaded
Upload Page Displayed	User uploads unsupported format	Upload Page (Error)
Upload Page (Error)	User retries with valid image	Image Uploaded

5.2.4.7 Image processing

Table 5.75: State transition Testing Image processing

State	Event	Transition
Image Uploaded	System starts processing	Processing In Progress
Processing In Progress	Processing completed	Processing Complete
Processing In Progress	Processing failed	Processing Failed

5.2.4.8 Visualize Results

Table 5.76: State transition Testing Visualize Results

State	Event	Transition
Processing Complete	System generates visual results	Results Visualized
Results Visualized	User closes viewer	Dashboard/Home

5.2.4.9 Recommendation

Table 5.77: State transition Testing Recommendation

State	Event	Transition
Results Visualized	System generates recommendation	Recommendation Shown
Recommendation Shown	User closes recommendation or proceeds	Dashboard/Home

5.2.5 Use Case Testing

5.2.5.1 User Register

Table 5.78: Use Case Testing User Register

Use Case	User Register
Actors	New User
Preconditions	User is not registered
Main Flow	Navigate → Enter details → Submit
Alternate Flow	Invalid inputs → Show errors
Expected Result	User is registered successfully

5.2.5.2 User Login

Table 5.79: Use Case Testing User Login

Use Case	User Login
Actors	Registered User
Preconditions	User is already registered
Main Flow	Navigate → Enter credentials → Login
Alternate Flow	Invalid credentials → Show login error
Expected Result	User is logged in and redirected

5.2.5.3 User Forget Password

Table 5.80: Use Case Testing User Forget Password

Use Case	Forget Password
Actors	Registered User
Preconditions	User forgets password
Main Flow	Click reset → Enter email → Get token → Reset
Alternate Flow	Invalid email/token → Show error
Expected Result	Password is reset successfully

5.2.5.4 User Logout

Table 5.81: Use Case Testing User Logout

Use Case	Logout
Actors	Logged-in User
Preconditions	User is logged in
Main Flow	Click logout → Session ends
Alternate Flow	N/A
Expected Result	User is logged out

5.2.5.5 User Connects to Database

Table 5.82: Use Case Testing User Connects to Database

Use Case	Connect to Database
Actors	System
Preconditions	App is running
Main Flow	System initiates DB connection
Alternate Flow	Connection fails → Retry/Error logged
Expected Result	System is connected to database

5.2.5.6 User Upload image

Table 5.83: Use Case Testing User Upload image

Use Case	Upload Image
Actors	User
Preconditions	User is logged in
Main Flow	Navigate → Choose file → Upload
Alternate Flow	Invalid format → Show error
Expected Result	Image is uploaded successfully

5.2.5.7 Image processing

Table 5.84: Use Case Testing Image processing

Use Case	Image Processing
Actors	System
Preconditions	Valid image uploaded
Main Flow	Start processing Analyze image
Alternate Flow	Corrupted image Processing error
Expected Result	Analysis completed

5.2.5.8 Visualize Results

Table 5.85: Use Case Testing Visualize Results

Use Case	Visualize Results
Actors	User
Preconditions	Image processed successfully
Main Flow	Display graphical result
Alternate Flow	Visualization fails Show message
Expected Result	Results shown visually

5.2.5.9 Recommendation

Table 5.86: Use Case Testing Recommendation

Use Case	Recommendation
Actors	System, User
Preconditions	Detection results are ready
Main Flow	Analyze disease Fetch treatment info
Alternate Flow	Missing data Show "No recommendations"
Expected Result	Recommendations are displayed

5.3 White Box Test Cases

5.3.1 Cyclometric complexity

Cyclomatic complexity measures the number of independent execution paths in the source code. Lowering this complexity enhances testability, reliability, and maintainability. The following analysis summarizes the complexity per requirement and outlines strategies to reduce it.

5.3.1.1 User Register

- **Complexity:** 6
- **Reasons:** Multiple validation checks (email, password, confirmation), uniqueness check, database write.
- **Recommendations:**
 - Modularize field validation into reusable utility functions.
 - Use early returns to handle invalid input.

5.3.1.2 User Login

- **Complexity:** 4
- **Reasons:** Input validation, authentication, session/token generation.
- **Recommendations:**
 - Simplify conditional logic.

Extract session/token logic into a separate service.

5.3.1.3 Forget Password

- **Complexity:** 5
- **Reasons:** Email verification, token generation, reset URL dispatch.
- **Recommendations:**
 - Centralize error handling.
 - Refactor reset logic into separate modules.

5.3.1.4 User Logout

- **Complexity:** 2
- **Reasons:** Basic session/token destruction.
- **Recommendations:**
 - Maintain simplicity.
 - Ensure consistent error handling.

5.3.1.5 User Connects to Database

- **Complexity:** 3
- **Reasons:** Connection string validation, exception handling.
- **Recommendations:**
 - Use retries strategies through wrapper functions.
 - Avoid deep nesting via early exits on failure.

5.3.1.6 Upload Image

- **Complexity:** 4
- **Reasons:** Format/type check, size check, upload response handling.
- **Recommendations:**
 - Create reusable validators.
 - Use centralized logging and exception handlers.

5.3.1.7 Image Processing

- **Complexity:** 7
- **Reasons:** Null check, preprocessing steps, ML model inference, exception catching, retry logic.
- **Recommendations:**
 - Decompose pipeline stages into separate services.
 - Use functional composition or decorators for filters.

5.3.1.8 Visualize Results

- **Complexity:** 3
- **Reasons:** Data validation, chart type decision.
- **Recommendations:**
 - Use default templates and standard chart configs to reduce branching.

5.3.1.9 Recommendation

- **Complexity:** 6
- **Reasons:** Rule engine, disease classification, fallback scenarios.
- **Recommendations:**
 - Replace nested if-else blocks with strategy pattern or rule engine.

Leverage polymorphism for recommendation logic.

5.4 Performance testing

5.4.1 User Registration

Performance testing for the user registration module ensures that the system can handle high volumes of user sign-ups without performance degradation. Load testing will simulate hundreds of concurrent users attempting to register at the same time, ensuring that the registration process completes within acceptable response times. Stress testing will gradually increase the number of users until the system fails, helping to identify the upper capacity limits. Spike testing will check how the system handles sudden surges in registration attempts.

5.4.2 User Login

For the login module, performance testing will measure response times and server behavior under concurrent login requests. The system should authenticate hundreds or thousands of users simultaneously without timeout issues. Testing will also assess session creation, maintenance, and expiration under load. Repeated login attempts by a single user over a period will test for memory leaks and stability in authentication mechanisms.

5.4.3 Forget Password

This module requires performance tests to verify that password reset emails are sent quickly and reliably under high traffic. Load testing will simulate a high volume of password reset requests to ensure the backend can generate tokens and send emails without delays. Rate-limiting will be verified by simulating multiple invalid reset attempts from the same user or IP to prevent abuse.

5.4.4 User Logout

Performance tests on the logout function ensure session termination is quick and reliable. Under concurrent logouts by many users, the system should clean up session data efficiently without lingering memory usage. Tests will ensure that users cannot continue accessing resources after logging out, even under heavy loads.

5.4.5 Connects to Database

Database connection performance is critical to the backend operations. Testing will involve measuring how many simultaneous connections the database can handle before latency increases. Additionally, tests will check how quickly typical queries (such as fetching user data or disease analysis records) are executed under load, ensuring all responses remain under 1 second for optimal performance.

5.4.6 Upload Image

Image upload performance testing involves uploading files of various sizes (e.g., 1MB to 100MB) to verify upload speed and success rate. Under heavy load with multiple users uploading simultaneously, the system should maintain responsiveness without crashes or significant delays. Timeout handling and file size validation will also be tested.

5.4.7 Image Processing

The core module of the system, image processing performance, must handle real-time analysis of wheat disease images. The system will be tested with a high volume of image inputs to measure how many can be processed per second. The goal is to ensure that each image is processed and classified in under one second, even when queued with hundreds of others. Memory consumption and CPU usage will be monitored to ensure system stability.

5.4.8 Visualize Results

Visualization performance tests will focus on how quickly graphs and charts load and update on the UI. When users view results, rendering should be smooth even with large datasets or multiple graphs being generated at once. Tests will simulate scenarios like 100 users simultaneously viewing analysis charts, checking for client-side rendering delays and server throughput.

5.4.9 Recommendation System

The recommendation engine must respond rapidly to analysis results, offering prevention or treatment suggestions. Performance tests will verify that under continuous requests (e.g., 1000 per second), the system maintains a sub-500ms response time. The system will also be tested for recommendation accuracy and consistency under high load, ensuring it doesn't degrade due to memory or CPU bottlenecks.

5.5 Stress Testing

5.5.1 User Registration

Stress testing this feature involves pushing the registration endpoint with far more simultaneous requests than expected under normal conditions. For example, simulate 10,000 concurrent users trying to register at once. The goal is to monitor how the system handles a surge: Does it slow down? Do requests fail gracefully? Is there a delay in email verification delivery? System logs, response time spikes, and database write behavior will be closely observed.

5.5.2 User Login

To stress test the login module, simulate continuous login attempts from thousands of users—especially repetitive logins from the same accounts (bot behavior simulation). Watch for signs of server CPU/memory overload, session token failures, or authentication service collapse. Also verify account lockout mechanisms and rate-limiting features to ensure they activate under brute force-like conditions.

5.5.3 Forget Password

Here, stress testing involves simulating mass requests for password resets, such as 5,000 users requesting password resets within a minute. The email dispatch service and token generation logic should be stress-tested for latency, accuracy, and delivery failure under pressure. Ensure the system avoids overloading mail queues or API rate limits from email providers.

5.5.4 User Logout

Although lightweight, logout functionality must still be tested under massive concurrent usage. Stress testing this involves triggering logout actions from tens of thousands of users within seconds. Observe how quickly sessions are terminated and whether user data is cleared efficiently. The system should remain responsive without consuming excess memory from lingering sessions.

5.5.5 Connects to Database

Stress testing the database connection pool involves simulating an extremely high number of concurrent queries, reads, and writes such as during mass user activity (e.g., login storms or image uploads). The test identifies max connection limits, query timeouts, and deadlock behavior. Use tools like Apache JMeter or Locust to simulate 100,000+ simultaneous read/write operations.

5.5.6 Upload Image

For this module, stress testing simulates thousands of users uploading large images (e.g., 10MB 50MB each) concurrently. Monitor file upload timeouts, server disk usage, memory buffer overflow, and bandwidth saturation. The objective is to find the limit at which the server or network cannot handle additional uploads and to ensure failure messages are properly handled.

5.5.7 Image Processing

The AI model backend is pushed with thousands of image processing requests back-to-back. This tests GPU/CPU utilization under stress, memory handling for model inference, and whether batch processing or queuing systems behave reliably. Stress tests may involve repeatedly sending corrupted or malformed images to test how robustly the model handles errors under load.

5.5.8 Visualize Results

Stress testing here involves having many users simultaneously load charts, filter results, or zoom through large visual datasets. Browser rendering and backend data aggregation will be monitored. The aim is to determine whether visual components lag, crash, or freeze, particularly under pressure from concurrent queries and visual rendering.

5.5.9 Recommendation System

The system is stress-tested by simulating hundreds of thousands of recommendation requests in a short span. This determines how well the system performs under extreme inference load, particularly when paired with real-time results from the image analysis pipeline. Stress testing also checks how gracefully the system handles queue buildup or recommendation delays.

5.6 System Testing

5.6.1 User Registration

- **Objective:** Validate new user account creation.
- **Test Actions:**
 - Open registration form.
 - Enter valid/invalid details.
 - Check for duplicate email.
 - Submit and verify account creation and database update.
- **Expected Result:** User is created, confirmation message is shown, email is sent.

5.6.2 User Login

- **Objective:** Verify that registered users can log in.
- **Test Actions:**
 - Input correct/incorrect credentials.
 - Attempt login with deactivated account.
 - Check token/session creation.
- **Expected Result:** Successful login creates a session and navigates to the dashboard.

5.6.3 Forget Password

- **Objective:** Ensure users can reset passwords securely.
- **Test Actions:**
 - Enter a valid/invalid email.
 - Receive reset link.
 - Reset password and log in.
- **Expected Result:** Link sent, password updated, login successful with new password.

5.6.4 User Logout

- **Objective:** Confirm session ends correctly.
- **Test Actions:**
 - Login and click logout.
 - Attempt to revisit authenticated page.

Expected Result: Session terminated; access blocked after logout.

5.6.5 Connects to Database

- **Objective:** Verify system successfully connects to DB on all modules.
- **Test Actions:**
 - Perform all CRUD operations through frontend.
 - Test reconnection if DB is temporarily unavailable.
- **Expected Result:** Seamless connection without crash, retry logic works.

5.6.6 Upload Image

- **Objective:** Test image upload functionality and validation.
- **Test Actions:**
 - Upload valid/invalid (wrong type, oversized) files.
 - Confirm upload triggers processing.
- **Expected Result:** Images accepted or rejected based on format; stored for analysis.

5.6.7 Image Processing

- **Objective:** Ensure image goes through AI model pipeline correctly.
- **Test Actions:**
 - Upload image and wait for classification.
 - Check result logs and UI output.
- **Expected Result:** Disease accurately identified; system returns result in real-time.

5.6.8 Visualize Results

- **Objective:** Display results in graphical format.
- **Test Actions:**
 - Navigate to results page.
 - Trigger chart generation.
 - Try filters and sorting options.
- **Expected Result:** Graphs render correctly, data points are accurate and interactive.

5.6.9 Recommendation

- **Objective:** Generate accurate preventive and corrective actions.
- **Test Actions:**
 - Trigger recommendations based on image results.
 - Check accuracy and relevance of content.
- **Expected Result:** System returns tailored suggestions based on disease severity/type.

5.7 Regression Testing

5.7.1 User Registration

- **Changes That Trigger Regression:**
 - Modified password validation rules.
 - Updated email verification flow.
- **What to Re-test:**
 - Field validation.
 - Email uniqueness check.
 - Account creation flow.

DB entry integrity

5.7.2 User Login

- **Changes That Trigger Regression:**
 - Updated token/session mechanism.
 - Added captcha or 2FA.
- **What to Re-test:**
 - Login with valid/invalid credentials.
 - Session generation and expiry.

Redirection after login.

5.7.3 Forget Password

- **Changes That Trigger Regression:**
 - Changed email service provider or email format.
- **What to Re-test:**
 - Reset request submission.
 - Email content and link.

Password update and reuse prevention.

5.7.4 User Logout

- **Changes That Trigger Regression:**
 - Changes to session/cookie handling.
- **What to Re-test:**
 - Session termination.
 - Blocked access after logout.
 - UI redirection.

5.7.5 Connect to Database

- **Changes That Trigger Regression:**
 - Switched DB engine or altered schema.
- **What to Re-test:**
 - CRUD operations for all modules.
 - Foreign key relationships.

Performance under multiple reads/writes.

5.7.6 Upload Image

- **Changes That Trigger Regression:**
 - Supported file types or size limits updated.
 - New progress bar or uploader UI.
- **What to Re-test:**
 - Image file validation.
 - Upload response time.
 - Storage location consistency.

5.7.7 Image Processing

- **Changes That Trigger Regression:**
 - Updated AI model or image preprocessor.
- **What to Re-test:**
 - Model input/output consistency.
 - Accuracy compared to previous versions.

Error handling on failed predictions.

5.7.8 Visualize Results

- **Changes That Trigger Regression:**
 - UI/UX enhancements or chart library changes.
- **What to Re-test:**
 - Graph rendering.
 - Filters and options.

Backend data mapping to charts.

5.7.9 Recommendation

- **Changes That Trigger Regression:**
 - Improved disease severity analysis.
 - New rule engine or ML model for suggestions.
- **What to Re-test:**
 - Relevance of suggestions.
 - Response time.
 - Linkage with the disease result.

CHAPTER 6

Tool and Techniques

6.1 Applications and Tools

6.1.1 Visual Studio Code (VSCode)

The primary integrated development environment (IDE) used for writing, testing, and debugging code. Along with features like syntax highlighting, debugging, version control, and extensions, it supports a large number of programming languages and frameworks. It is appropriate for both front-end and back-end development because it supports a variety of languages.

6.1.2 Visual Paradigm

A wide range of diagramming types, including UML, ERD, flowcharts, and others, are supported by Visual Paradigm, a visual modeling and design tool. It facilitates the design and visualization of databases, processes, and software architecture. We have used it for designing and documenting the Property Expertise Hub's architecture, it is utilized to create flow diagrams, UML diagrams (such as use case, sequence, and class diagrams), and other visual models. It facilitates comprehension of the system architecture and guarantees rapid deployment.

6.1.3 Google Colab

Google Colab is a cloud-based Jupyter Notebook environment that provides access to powerful GPUs and TPUs. It enabled you to train your wheat leaf detection model without needing local setup or expensive hardware. Colab allows you to write and execute Python code in your browser, making it ideal for machine learning projects and easy sharing of your work.

6.2 Techniques

6.2.1 Machine Learning Models

Applied for analyzing uploaded crop images and accurately detecting rust diseases using trained image classification algorithms. A machine learning model is a representation of a real-world process that allows computers to make predictions or decisions based on data. These models learn patterns from historical data and apply them to new inputs without being explicitly programmed.

6.2.2 React.js

Employed to create a responsive and modular frontend for the web application. React is a popular JavaScript library for building interactive user interfaces, particularly for single-page applications. It allows developers to create reusable UI components and manage state efficiently using a virtual DOM.

6.2.3 JavaScript (React.js)

Used for developing the user interface and client-side interactions. JavaScript is the backbone of React.js, a library designed for building dynamic, high-performance user interfaces. Since React is declarative and component-based, it simplifies the creation of complex UI elements.

6.2.4 AOS (Animate on Scroll)

Used to implement animation effects that enhance user experience by triggering animations as elements scroll into view. It's widely used for adding smooth animations to web pages without requiring complex CSS or JavaScript.

6.2.5 React Icons

A lightweight icon library used for embedding scalable and customizable icons in the user interface. It allows developers to easily integrate icons from various sources, such as Font Awesome, Material Design, and more, using simple ES6 imports.

6.2.6 Python

Python, a high-level programming language, was the primary language for developing this project. Its simplicity, extensive libraries, and ecosystem allowed for seamless development of web applications, machine learning integration, and API management.

6.2.7 Machine Learning Integration

A pre-trained deep learning model was loaded and deployed using TensorFlow to classify plant diseases based on image inputs. The model's predictions help users identify plant health issues and take timely actions.

6.3 Framework

6.3.1 Django

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

6.3.2 Engine TensorFlow

TensorFlow was used to load and deploy a pre-trained machine learning model for plant disease prediction. This deep learning framework supports the image classification functionality of the system, enabling accurate diagnoses of plant diseases. TensorFlow not only powers the deep learning model used for disease prediction but also provides the tools for model training and evaluation. The pre-trained model is deployed for real-time plant disease classification based on uploaded images.

CHAPTER 7

Summary and Conclusion

7.1 Summary

SmartRust AI is a web-based intelligent system designed to detect and help prevent wheat leaf diseases, primarily yellow rust, brown rust, and black rust. These diseases are a major threat to global wheat production, particularly affecting small holder farmers who may lack access to expert diagnosis. The goal of this project is to provide an accessible, automated solution for early disease detection using machine learning and image processing techniques. The platform allows users to register, log in, and upload images of diseased wheat leaves. These images are processed by a trained machine learning model, which accurately classifies the disease and generates corresponding treatment recommendations. The frontend of the system is built using React.js for a responsive and interactive user experience, while the backend uses Python and stores data in PostgreSQL 17. Technologies such as AOS (Animate on Scroll) and React Icons were used to enhance user interface aesthetics and interactivity. The system uses the InceptionV3 model for image recognition and classification, ensuring high accuracy in identifying wheat leaf diseases. TensorFlow is the machine learning framework used to build, train, and deploy models like InceptionV3. Testing played a critical role in validating the system's reliability. Black box testing, boundary value analysis, equivalence partitioning, and decision table testing were conducted to verify the correct functioning of features such as user login, image upload, disease processing, and result visualization. Both valid and invalid inputs were tested across all modules to ensure robustness and error handling. SmartRust AI serves as a proof of concept that demonstrates the power of integrating AI into agriculture. It provides a fast, cost-effective, and easy to use tool for farmers to diagnose diseases without relying on experts. The system has strong potential for future expansion, including mobile app development, support for multiple crops, integration with weather data, and predictive disease modeling. This project not only fulfills a practical need but also showcases the real-world impact of combining artificial intelligence with agricultural science to support sustainable farming. This project has enhanced our understanding of how modern technologies like AI and machine learning can be used to solve real-world agricultural problems. It improved our practical skills in software development, system testing, and user interface design. SmartRust AI lays the groundwork for future innovations in smart farming. We believe this solution has the potential to make a meaningful difference in the lives of farmers worldwide.

7.2 Conclusion

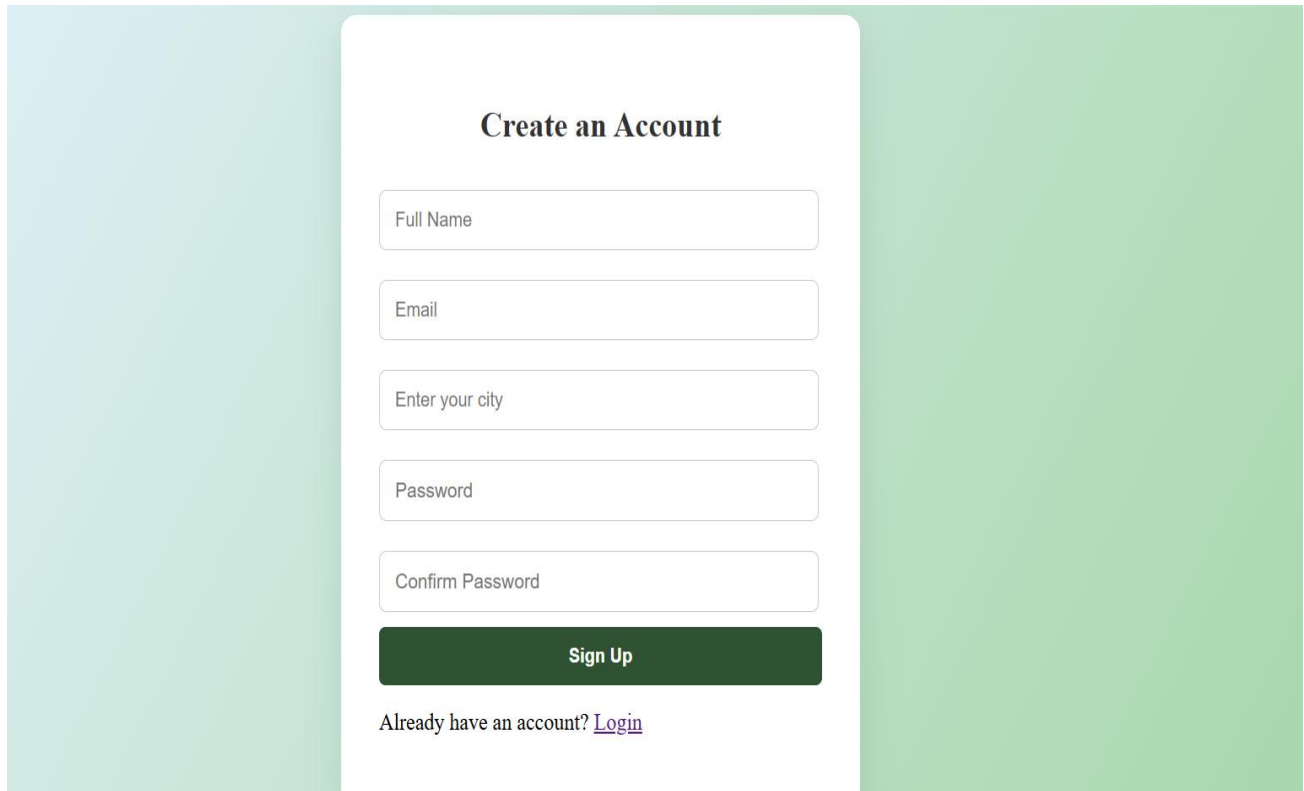
SmartRust AI successfully demonstrates how artificial intelligence can be applied to solve real-world agricultural problems. By automating the detection of wheat leaf diseases, the system offers a reliable, scalable, and easy-to-use solution for farmers, especially those without access to expert support. The integration of modern web technologies with machine learning ensures both usability and accuracy, making the platform suitable for real-time field applications. The project has not only met its technical and functional goals but also contributed to broader objectives such as supporting sustainable farming, reducing crop loss, and promoting the adoption of digital tools in agriculture. With further enhancements and field testing, SmartRust AI has the potential to evolve into a vital tool in precision agriculture and serve as a blueprint for similar solutions across other crops and regions.

CHAPTER 8

User Manual

8.1 User Register

1. To create a new account:
2. Open the SmartRust website.
3. Click on Register.
4. Fill in your Name, Email, city and Password.
5. Click Submit to create your account.

A user registration form titled "Create an Account" is centered on a light green background. The form is contained within a white rounded rectangle. It features five input fields: "Full Name", "Email", "Enter your city", "Password", and "Confirm Password". Below these fields is a dark green "Sign Up" button. At the bottom of the form, there is a link that says "Already have an account? [Login](#)".

Create an Account

Full Name

Email

Enter your city

Password

Confirm Password

Sign Up

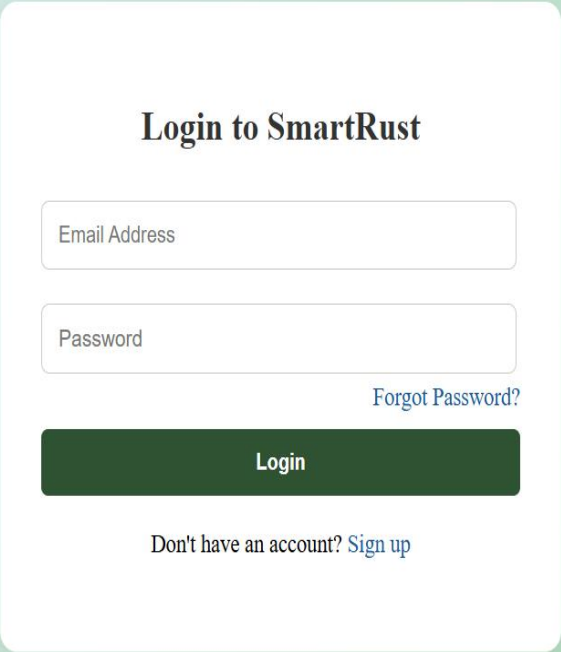
Already have an account? [Login](#)

Figure 8.42 User Register

8.2 User Login

To log into your account:

1. Navigate to the Login screen.
2. Enter your registered Email and Password.
3. Click Login.

The image shows a user login form for SmartRust. The form is centered on a light green background. It has a title "Login to SmartRust" in bold. Below the title are two input fields: "Email Address" and "Password". To the right of the password field is a link "Forgot Password?". Below these fields is a dark green "Login" button. At the bottom of the form, there is a link "Don't have an account? Sign up".

Login to SmartRust

Email Address

Password

[Forgot Password?](#)

Login

Don't have an account? [Sign up](#)

Figure 8.43 User Login

8.2 User Forget Password

1. If you can't access your account:
2. Click Forget Password on the login screen.
3. Enter your registered email.
4. Check your inbox for the reset link.

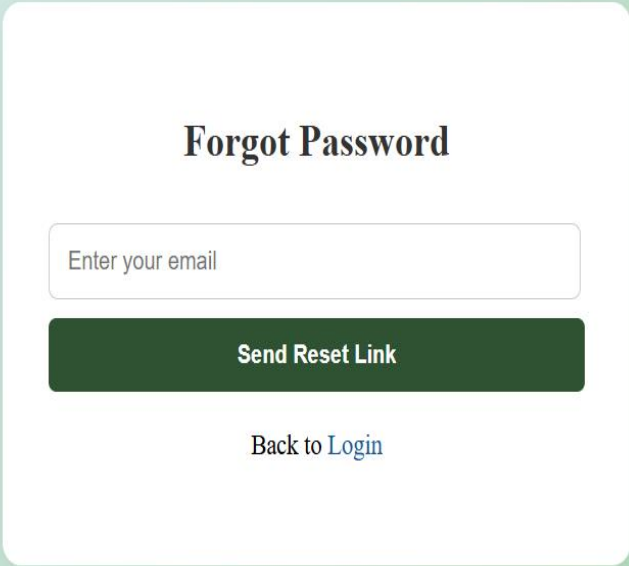
A screenshot of a 'Forgot Password' form. The form is a white rounded rectangle centered on a green gradient background. At the top of the form, the text 'Forgot Password' is displayed in a bold, black, serif font. Below this, there is a text input field with the placeholder text 'Enter your email'. Underneath the input field is a dark green button with the text 'Send Reset Link' in white. At the bottom of the form, there is a link that says 'Back to Login', where 'Login' is a blue hyperlink.

Figure 8.44 User Forget Password

8.2 Dashboard Overview

1. Once logged in, the dashboard gives you access to all system functionalities:
2. Upload Image: Submit wheat leaf photos for analysis.
3. Image Analysis: Launch disease detection algorithms.
4. Statistics: visual summaries.
5. Recommendations: Receive suggestions for disease treatment and field management.

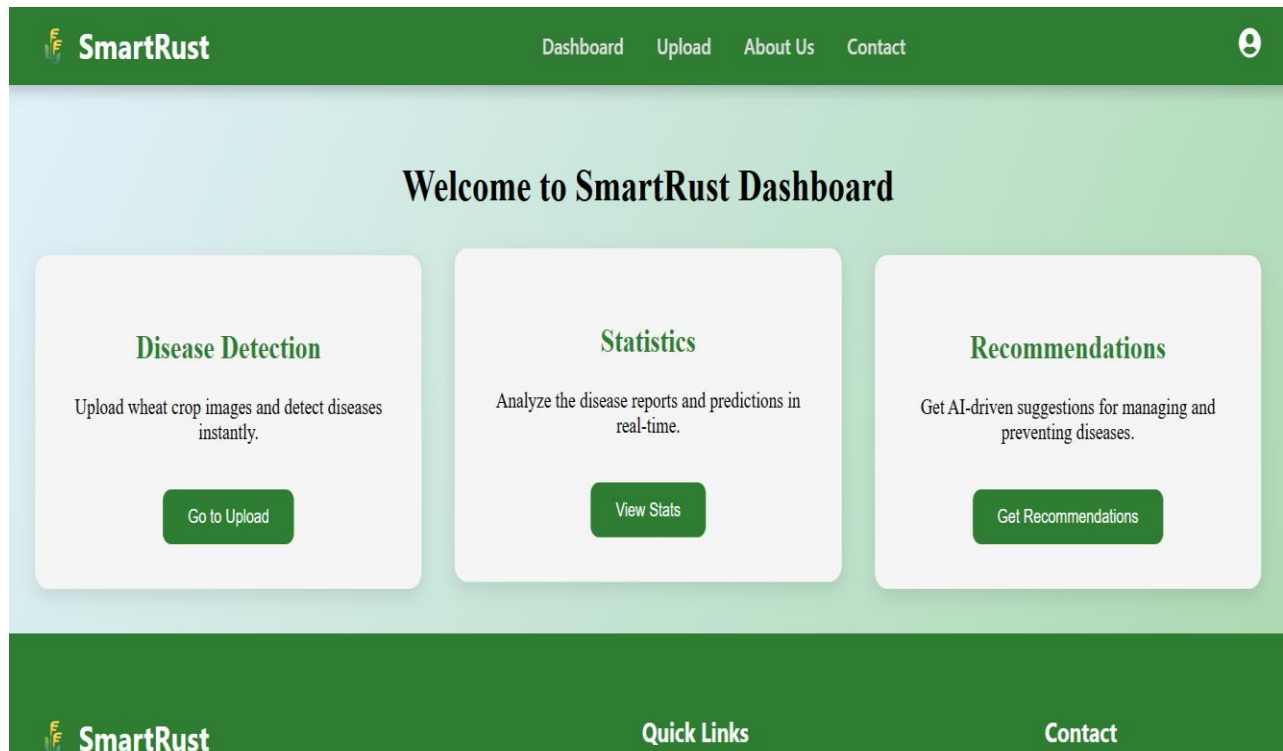


Figure 8.45 Dashboard Overview

8.3 Uploading Images

To upload an image:

1. Go to the Upload Image section.
2. Click Browse or Drag & Drop your image.
3. Ensure the file format is JPG or PNG and size is under 10MB.
4. Click Submit.

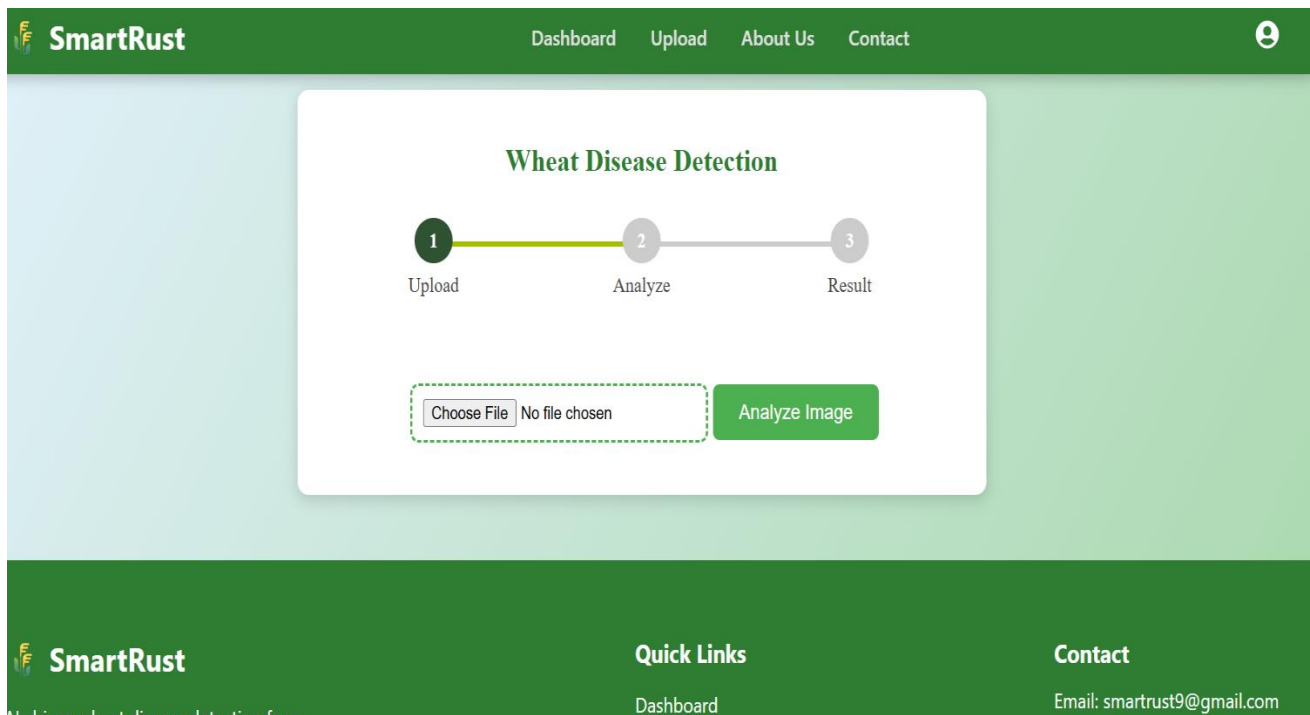


Figure 8.46 Dashboard Overview

8.4 Analyzing Images

After uploading:

1. Navigate to Image Analysis.
2. Select the image to analyze.
3. Click Analyze.
4. Wait while SmartRust processes the image.

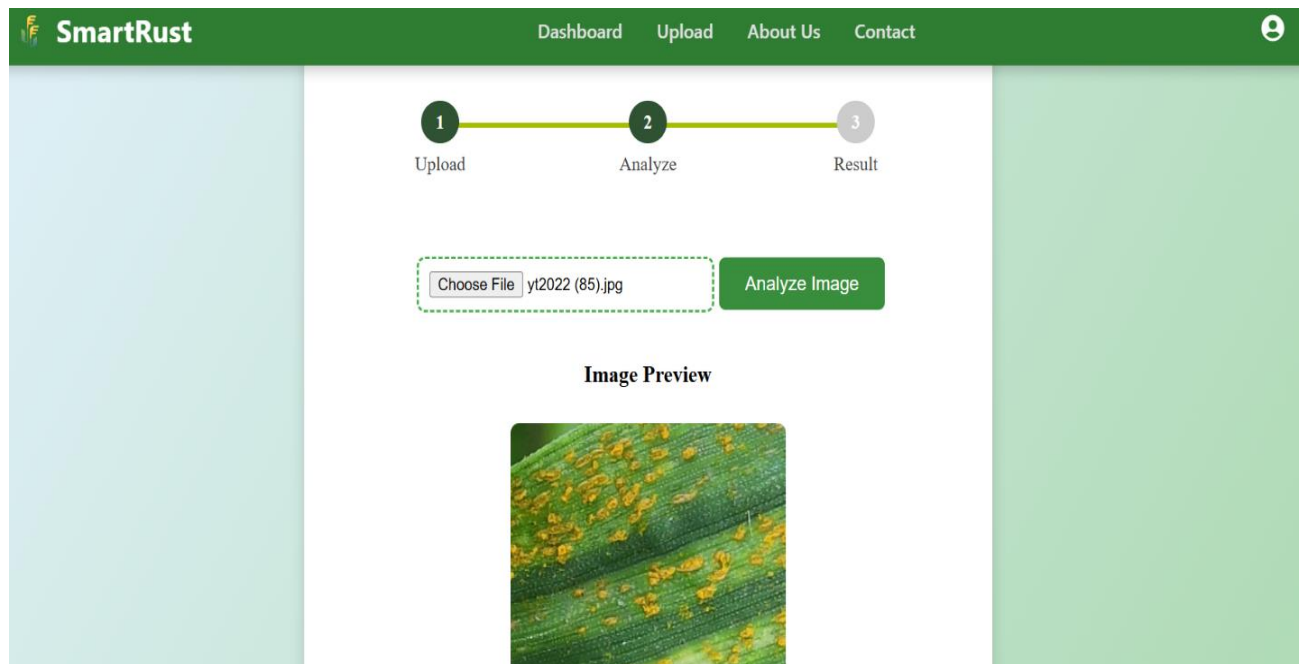


Figure 8.47 Analyzing Images

8.5 Viewing Results

To see the analysis:

1. Open the Result section.
2. Review the type of disease detected.
3. Graphs and stats are available in the Statistics tab.

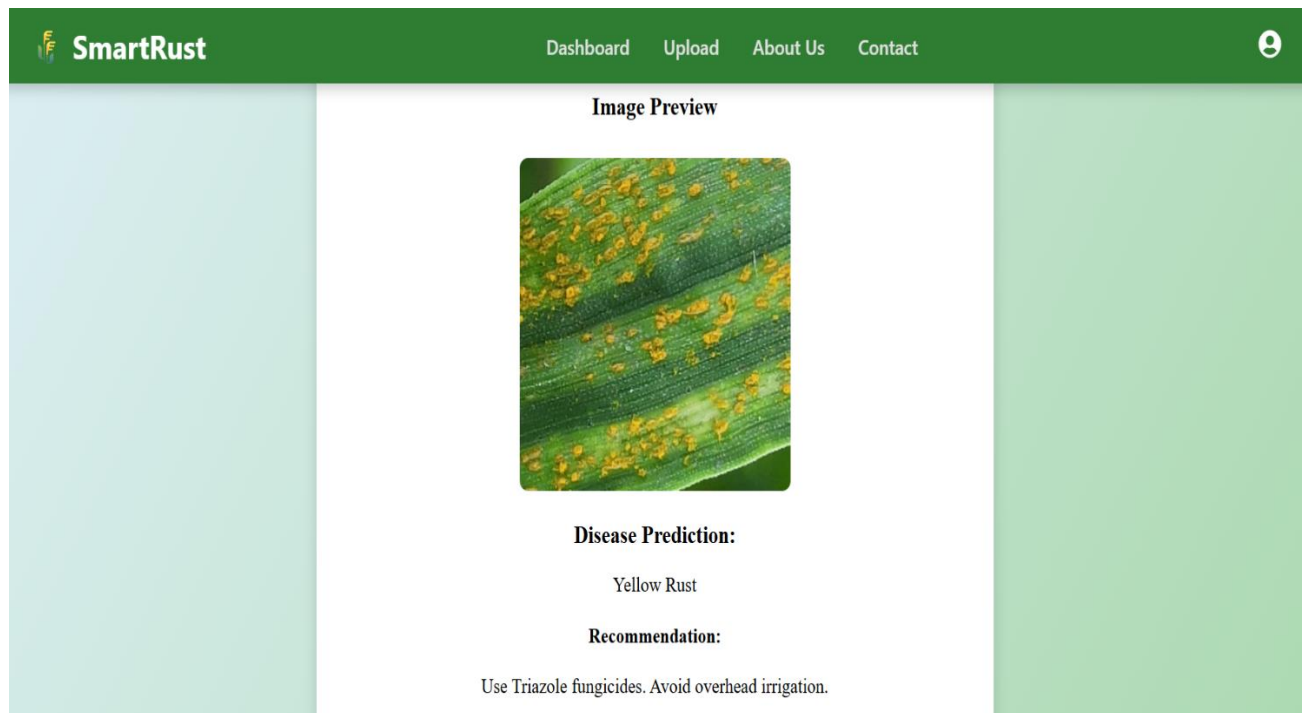


Figure 8.48 Viewing Results

8.6 Recommendations

Click Recommendations to see treatment steps, fungicide suggestions, and prevention tips
SmartRust provides customized advice based on disease type:

1. Brown Rust
2. Black Rust
3. Yellow Rust

Also includes general field guidance:

4. Soil Health Management
5. Weather Monitoring
6. Timely Field Inspections

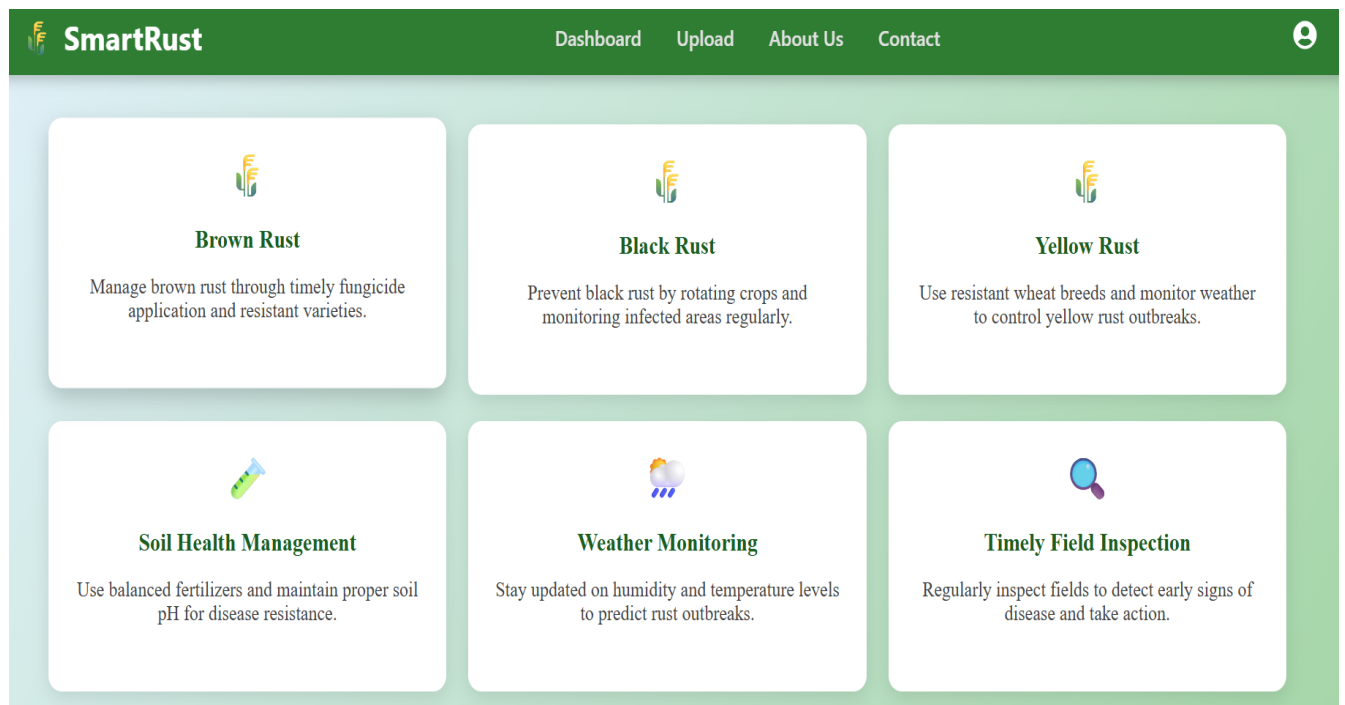


Figure 8.49 Recommendations

8.7 About Us

Learn more about the team and vision behind SmartRust .

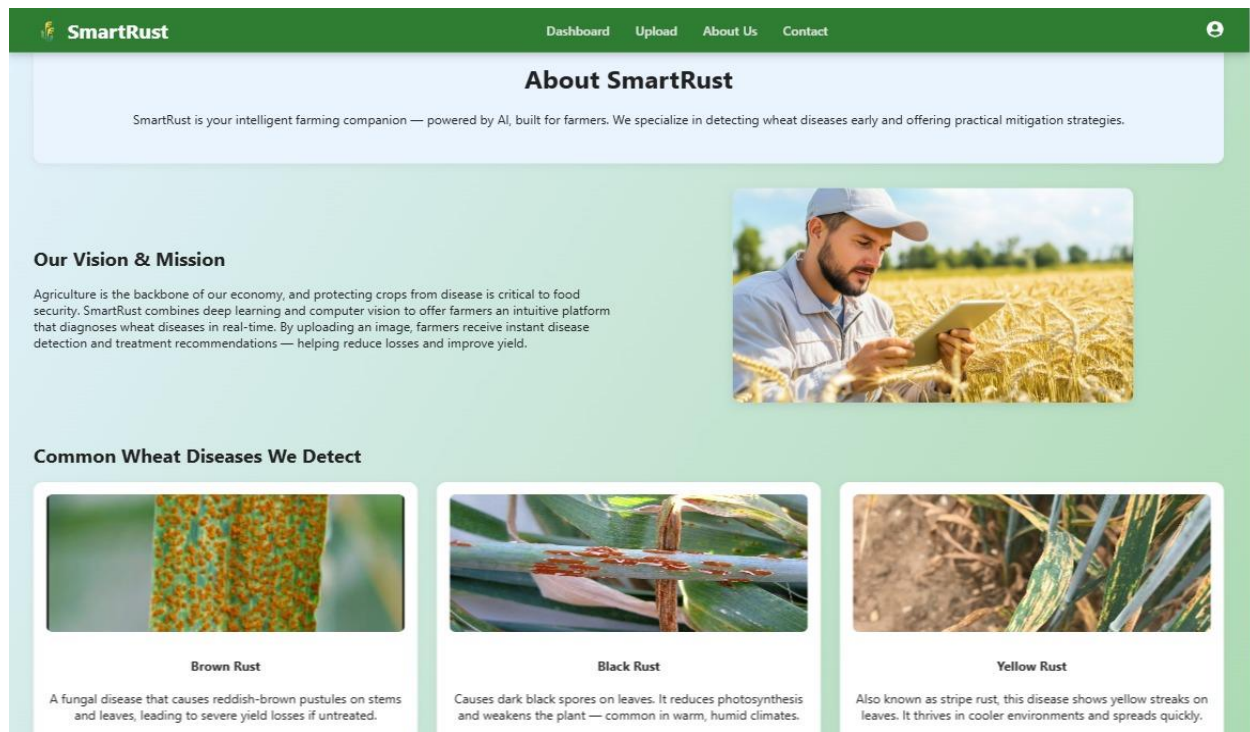


Figure 8.50 About Us

8.8 Contact Us

Reach out for support, feedback, or collaboration inquiries.

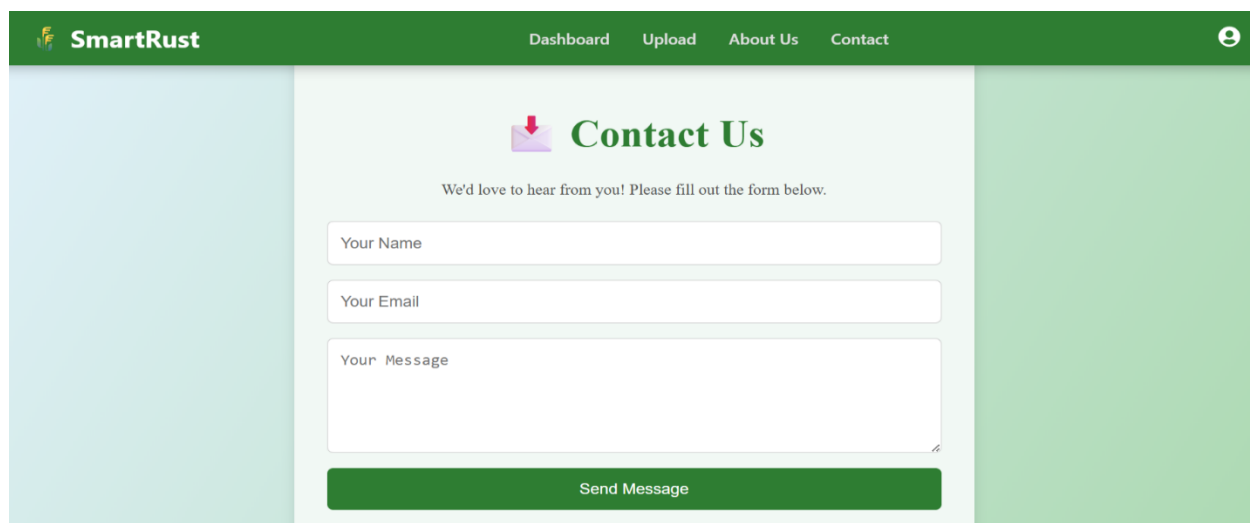


Figure 8.51 Contact Us

References

- Abade, A., P.A. Ferreira and F. de Barros Vidal. 2021. Plant diseases recognition on images using convolutional neural networks: A systematic review. *Comput. Electron. Agric.* 185:106125.
- Agarwal, M., A. Singh, S. Arjaria, A. Sinha and S. Gupta. 2020. ToLeD: Tomato Leaf Disease Detection using Convolution Neural Network. *Procedia Comput. Sci.* 167:293–301.
- Ahmed, M.R. 2021. Leveraging convolutional neural network and transfer learning for cotton plant and leaf disease recognition. *Int. J. Image Graph. Signal Process.* 13:47–62.
- Amorim, W.P., E.C. Tetila, H. Pistori and J.P. Papa. 2019. Semi-supervised learning with convolutional neural networks for UAV images automatic recognition. *Comput. Electron. Agric.* 164:104932.
- Ansari, A.S., M. Jawarneh, M. Ritonga, P. Jamwal, M.S. Mohammadi, R.K. Veluri, V. Kumar and M.A. Shah. 2022. Improved Support Vector Machine and Image Processing Enabled Methodology for Detection and Classification of Grape Leaf Disease. *J. Food Qual.* 2022.
- Anubha Pearline, S., V. Sathiesh Kumar and S. Harini. 2019. A study on plant recognition using conventional image processing and deep learning approaches. *J. Intell. Fuzzy Syst.* 36:1997–2004.
- Ashourloo, D., H. Aghighi, A.A. Matkan, M.R. Mobasheri and A.M. Rad. 2016. An Investigation Into Machine Learning Regression Techniques for the Leaf Rust Disease Detection Using Hyperspectral Measurement. *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.* 9:4344–4351.
- Atila, Ü., M. Uçar, K. Akyol and E. Uçar. 2021. Plant leaf disease classification using EfficientNet deep learning model. *Ecol. Inform.* 61:101182.
- Atta, B.M., M. Saleem, H. Ali, M. Bilal and M. Fayyaz. 2020. Application of Fluorescence Spectroscopy in Wheat Crop: Early Disease Detection and Associated Molecular Changes. *J. Fluoresc.* 30:801–810.
- Azadbakht, M., D. Ashourloo, H. Aghighi, S. Radiom and A. Alimohammadi. 2019. Wheat leaf rust detection at canopy scale under different LAI levels using machine learning techniques. *Comput. Electron. Agric.* 156:119–128.
- Guan, X. 2021. A novel method of plant leaf disease detection based on deep learning and convolutional neural network. In: 2021 6th International Conference on Intelligent Computing and Signal Processing (ICSP), IEEE, 816–819.
- Guo, A., W. Huang, H. Ye, Y. Dong, H. Ma, Y. Ren and C. Ruan. 2020a. Identification of wheat yellow rust using spectral and texture features of hyperspectral images. *Remote Sens.* 12:1419.
- Guo, Y., J. Zhang, C. Yin, X. Hu, Y. Zou, Z. Xue and W. Wang. 2020b. Plant disease identification based on deep learning algorithm in smart farming. *Discret. Dyn. Nat. Soc.* 2020:1–11.
- Gupta, S., J. Vishnoi and A.S. Rao. 2022. Disease Detection in Maize Plant using Deep Convolutional Neural Network. In: 2022 7th International Conference on Communication

- and Electronics Systems (ICCES), IEEE, 1330–1335.
- Holzinger, A., B. Malle, A. Saranti and B. Pfeifer. 2021. Towards multi-modal causability with graph neural networks enabling information fusion for explainable AI. *Inf. Fusion*. 71:28–37.
- Jiang, P., Y. Chen, B. Liu, D. He and C. Liang. 2019. Real-time detection of apple leaf diseases using deep learning approach based on improved convolutional neural networks. *IEEE Access*. 7:59069–59080.
- Jiang, Z., Z. Dong, W. Jiang and Y. Yang. 2021. Recognition of rice leaf diseases and wheat leaf diseases based on multi-task deep transfer learning. *Comput. Electron. Agric.* 186:106184.
- Li, L., S. Zhang and B. Wang. 2021. Plant Disease Detection and Classification by Deep Learning - A Review. *IEEE Access*. 9:56683–56698.
- Li, X.L., Z.H. Ma, C. Giagnocavo, F. Qin, H.G. Wang and J.A. Álvarez-Bermejo. 2017. Development of automatic counting system for urediospores of wheat stripe rust based on image processing. *Int. J. Agric. Biol. Eng.* 10:134–143.
- Mishra, S., R. Sachan and D. Rajpal. 2020. Deep Convolutional Neural Network based Detection System for Real-time Corn Plant Disease Recognition. *Procedia Comput. Sci.* 167:2003–2010.
- Ngugi, L.C., M. Abelwahab and M. Abo-Zahhad. 2021. Recent advances in image processing techniques for automated leaf pest and disease recognition – A review. *Inf. Process. Agric.* 8:27–51.
- Nigam, S., R. Jain, S. Marwaha, A. Arora, M.A. Haque, A. Dheeraj and V.K. Singh. 2023. Deep transfer learning model for disease identification in wheat crop. *Ecol. Inform.* 75:102068.
- Novtahaning, D., H.A. Shah and J.-M. Kang. 2022. Deep Learning Ensemble-Based Automated and High-Performing Recognition of Coffee Leaf Disease. *Agriculture*. 12:1909.
- Ouhami, M., A. Hafiane, Y. Es-Saady, M. El Hajji and R. Canals. 2021. Computer vision, IoT and data fusion for crop disease detection using machine learning: a survey and ongoing research. *Remote Sens.* 13:2486.
- Waheed, A., M. Goyal, D. Gupta, A. Khanna, A.E. Hassanien and H.M. Pandey. 2020. An optimized dense convolutional neural network model for disease recognition and classification in corn leaf. *Comput. Electron. Agric.* 175:105456.
- Wang, P., T. Niu, Y. Mao, Z. Zhang, B. Liu and D. He. 2021. Identification of apple leaf diseases by improved deep convolutional neural networks with an attention mechanism. *Front. Plant Sci.* 12:723294.
- Zhang, J., L. yuan, R. Pu, R.W. Loraamm, G. Yang and J. Wang. 2014. Comparison between wavelet spectral features and conventional spectral features in detecting yellow rust for winter wheat. *Comput. Electron. Agric.* 100:79–87.